



UNIVERSITÀ DI TRENTO

Dipartimento di Ingegneria e Scienza dell'Informazione

Corso di Laurea in
Informatica

ELABORATO FINALE

TRADUZIONE TEXT-TO-SQL

Modello di traduzione di testo da linguaggio naturale a linguaggio SQL

Supervisore

Paolo Giorgini

Collaboratori

Marco Bombieri, Marco Robol

Laureando

Pietro Gasparini

Anno accademico 2025/2026

Sommario

L'interazione tra linguaggio naturale e database relazionali rappresenta una delle sfide più rilevanti nell'ambito dell'Intelligenza Artificiale applicata. Questa tesi presenta lo sviluppo di un framework avanzato di Text-to-SQL progettato per ottimizzare la generazione di query attraverso un approccio modulare e iterativo.

A differenza dei modelli tradizionali, il sistema implementato si basa su un'architettura a doppia orchestrazione: lo **SchemaOrchestrator**, dedicato alla ricostruzione semantica e all'acquisizione della struttura del database, e il **QueryOrchestrator**, responsabile della generazione e raffinamento del codice SQL. Un elemento distintivo della ricerca è l'integrazione di tecniche di Retrieval-Augmented Generation (RAG) tramite componenti di storage vettoriale (**SchemaStore** e **QueryStore**), che permettono al sistema di mantenere una memoria storica delle interrogazioni passate e dei relativi fallimenti.

Il cuore innovativo del lavoro risiede nel ciclo di feedback automatizzato: il sistema non si limita a produrre un output, ma esegue una validazione sintattica e semantica in tempo reale. In caso di errore, un modulo di analisi (**LLMFeedback**) categorizza l'anomalia, fornendo istruzioni mirate al modello per la correzione autonoma. I risultati dimostrano come l'uso combinato di prompt ingegnerizzati, contestualizzazione dinamica dello schema e apprendimento dagli errori pregressi aumenti significativamente l'affidabilità delle query generate, portando in media il 21,54% di successi in più, riducendo la distanza tra l'intento dell'utente e l'esecuzione tecnica su database eterogenei.

Uso di strumenti di Intelligenza Artificiale

Per lo sviluppo del progetto di tesi sono stati impiegati strumenti di Intelligenza Artificiale, in particolare durante la fase di elaborazione del codice è stato utilizzato ChatGPT [1] ed il suo tool integrato Codex [2] per organizzare la struttura e la generazione di alcune formule ausiliarie. Per quanto riguarda la stesura del testo, è stato sfruttato NotebookLLM [3] per la raccolta, gestione e traduzione delle fonti mentre l'utilizzo di Google Gemini [4] ha permesso di controllare la correttezza dal punto di vista grammaticale e ortografico del testo.

Indice

Sommario	1
1 Introduzione	5
1.1 Presentazione del problema	5
1.2 Obiettivi e Risultati Attesi	5
1.3 Struttura della Tesi	6
2 Stato dell'arte e Background	7
2.1 Recenti scoperte ed invenzioni	7
2.2 Metodologia Text-to-SQL e Integrazione del RAG	8
2.3 Approfondimento sui tool presenti in commercio	9
3 Soluzione proposta	10
3.1 Schema Orchestrator	10
3.1.1 Acquisizione dello schema	10
3.1.2 Aggiornamento dello schema	12
3.2 Query Orchestrator	12
3.2.1 Generazione e Raffinamento Iterativo della Query	12
3.2.2 Valutazione della query	13
3.3 Interfaccia	14
4 Test e valutazioni	15
4.1 Architettura del Sistema di Testing	16
4.2 Fase di analisi	18
4.2.1 Analisi Macro-Livello dei Database	20
4.2.2 Analisi Comparativa del Comportamento dei Modelli	21
4.2.3 Analisi Inferenziale e Studio delle Correlazioni Statistiche	25
4.2.4 Conclusioni dell'Analisi	28
5 Conclusioni	29
5.1 Raggiungimento degli obiettivi	29
5.2 Vantaggi e svantaggi	30
5.3 Prospettive future	31
Bibliografia	33
A Prompt Ingegnerizzati	35
A.1 Prompt per la Generazione dello Schema	35
A.2 Prompt per l'Aggiornamento dello Schema	35
A.3 Prompt per la Traduzione Text-to-SQL	36
A.4 Prompt per il Feedback dei Circuiti di Valutazione	37
A.5 Prompt Ausiliari e di Benchmarking	38
B Video Interfaccia	38
C Tabelle Statistiche	38
C.1 Tabelle di GPT-4o	39
C.1.1 Tabella Status	39
C.1.2 Tabella Attempts Correlations	39
C.1.3 Tabella Complexity Correlations	39
C.2 Tabelle di GPT-5-mini	39
C.2.1 Tabella Status	39

C.2.2	Tabella Attempts Correlations	39
C.2.3	Tabella Complexity Correlations	39
C.3	Tabelle di Codellama	39
C.3.1	Tabella Status	39
C.3.2	Tabella Attempts Correlations	39
C.3.3	Tabella Complexity Correlations	39
C.4	Tabelle di Codestral	40
C.4.1	Tabella Status	40
C.4.2	Tabella Attempts Correlations	40
C.4.3	Tabella Complexity Correlations	40
C.5	Tabelle di SQLCoder	40
C.5.1	Tabella Status	40
C.5.2	Tabella Attempts Correlations	40
C.5.3	Tabella Complexity Correlations	40
C.6	Tabelle di Qwen3-Coder-Next	40
C.6.1	Tabella Status	40
C.6.2	Tabella Attempts Correlations	40
C.6.3	Tabella Complexity Correlations	40
C.7	Tabella Database	40

1 Introduzione

Nell'attuale era digitale, i dati rappresentano la risorsa strategica centrale per ogni tipo di organizzazione, dalla sanità al commercio, fino allo sport professionistico. Nonostante i database relazionali (RDBMS) siano il motore dinamico della ricerca scientifica e del business, il loro accesso è ancora limitato dalla barriera tecnica del linguaggio SQL.

1.1 Presentazione del problema

La necessità di una sintassi rigorosa rende i dati accessibili quasi esclusivamente a figure specializzate, costringendo gli utenti non tecnici (manager, medici o analisti) a dipendere dai reparti IT. Questa intermediazione genera inefficienze critiche: rallentamenti decisionali, elevati costi di formazione e il rischio di incomprensioni tra chi pone la domanda e chi scrive il codice, lasciando spesso inespresso il reale potenziale del patrimonio informativo aziendale. La spinta principale di questo lavoro risiede nella democratizzazione dell'accesso ai dati. In un mercato che richiede risposte immediate, è essenziale trasformare i database da archivi statici in strumenti di conversazione aperti a tutti. Automatizzare la traduzione in SQL significa garantire autonomia decisionale e migliorare l'efficienza operativa: il valore di un'intuizione non deve più essere limitato dalla conoscenza del codice, ma solo dalla capacità umana di porre le domande giuste. Il settore del Text-to-SQL è stato rivoluzionato dai Large Language Models (LLM). Sebbene modelli generalisti come GPT-4 abbiano aperto la strada, l'attenzione si è oggi spostata su soluzioni specializzate come Qwen-Coder o modelli fine-tuned come SQLCoder, capaci di gestire schemi complessi con maggiore precisione. Parallelamente, l'introduzione della tecnica RAG (Retrieval-Augmented Generation) ha segnato un punto di svolta, permettendo al sistema di "recuperare" metadati e la documentazione per costruire prompt contestualizzati. Questo approccio riduce le allucinazioni dell'IA e permette ai modelli di adattarsi a database specifici senza necessità di riaddestramento. Tale evoluzione ha favorito la nascita di strumenti pronti all'uso come Text2SQL o AskYourDatabase, che iniziano a integrare queste tecnologie con interfacce intuitive per l'utente finale.

1.2 Obiettivi e Risultati Attesi

L'obiettivo principale di questo lavoro di tesi risiede nello sviluppo, nell'implementazione e nella validazione sperimentale di un sistema modulare a oggetti in linguaggio Python. Tale sistema ingegnerizzato deve tradurre in modo efficace e sintatticamente coerente richieste formulate in linguaggio naturale in interrogazioni SQL direttamente eseguibili su un DBMS target. Il nucleo del progetto si focalizza sulla creazione di uno strato software strutturato che opera come ponte tra l'utente (o un'applicazione client) e la complessità intrinseca degli schemi relazionali, garantendo un'elevata precisione nella mappatura tra i concetti espressi verbalmente e le strutture logiche delle tabelle.

L'intento è fornire uno strumento flessibile, disaccoppiato e facilmente integrabile in ecosistemi software preesistenti, abilitando piattaforme terze a implementare funzionalità di analisi dati in modalità self-service. Attraverso l'impiego di modelli di linguaggio avanzati (LLM) e di tecniche evolute di gestione del contesto, l'obiettivo finale dimostra la fattibilità dell'abbattimento della barriera tecnica del codice, trasformando l'interrogazione dei dati in un processo immediato, sicuro e accessibile.

Per conseguire questo traguardo, il lavoro di ricerca e sviluppo si articola in una serie di obiettivi intermedi focalizzati sull'automazione del workflow di traduzione, esecuzione e affinamento:

- **Estrazione e normalizzazione dello schema:** Automatizzare la mappatura del database target sia tramite connessione diretta e recupero dei metadati dizionari, sia attraverso l'induzione e la normalizzazione di input non strutturati (istruzioni DDL) in un formato JSON standardizzato, successivamente indicizzato all'interno di un database vettoriale.
- **Ottimizzazione del contesto:** Ridurre la finestra di calcolo (context window) dei modelli linguistici estraendo dinamicamente, tramite euristiche di recupero vettoriale, solo le entità strettamente necessarie all'intento espresso dall'utente, arricchendo lo schema con regole di business e metadati extrastrutturali.

- **Generazione tramite prompt dinamici:** Sviluppare un motore di compilazione dei prompt che aggrega algoritmicamente schemi relazionali, percorsi di giunzione (join paths) e lo storico dei fallimenti passati, garantendo il completo disaccoppiamento logico dai modelli linguistici sottostanti.
- **Validazione e raffinamento iterativo (Refinement):** Implementare un ciclo di feedback a ciclo chiuso che verifica a runtime la conformità sintattica ed esecutiva della query prodotta, intercettando le eccezioni sollevate dal database per guidare un processo autonomo di auto-correzione.
- **Esposizione architetturale tramite API:** Ingegnerizzare l'applicazione separando nettamente la logica di backend dal frontend, esponendo le funzionalità di generazione query e di supervisione critica attraverso endpoint programmabili e facilmente interrogabili.

Dal punto di vista prettamente ingegneristico, il progetto si concretizza nella realizzazione di un modulo API robusto e scalabile, distribuito sotto forma di un pacchetto Python documentato e pronto all'uso. Tale soluzione è concepita per essere integrata agevolmente in applicazioni esterne, quali dashboard di Business Intelligence o agenti conversazionali aziendali, dimostrando versatilità nel gestire schemi relazionali eterogenei.

Un risultato chiave del framework è rappresentato dall'ottimizzazione del contesto tramite metodologie RAG (*Retrieval-Augmented Generation*). L'efficacia del sistema di recupero e l'integrazione dinamica dello schema nel prompt consentono al modello di operare con successo anche su database non inclusi nella fase di addestramento iniziale, puntando ad ottenere una notevole capacità di generalizzazione in modalità zero-shot.

A completamento dello sviluppo, l'intera impalcatura software è sottoposta a una massiccia campagna di test basata sui principali benchmark scientifici del settore, quali Spider e BIRD. Attraverso di essi sono stati ottenuti risultati quantitativi e qualitativi che rivelano quali dei requisiti sono stati raggiunti dal framework. Il principale traguardo consiste nel raggiungimento di un'elevata accuratezza nella traduzione (Execution Accuracy), minimizzando i fenomeni di allucinazione dei modelli e garantendo che il codice SQL prodotto sia pronto per l'esecuzione sul database target senza interventi manuali. Di conseguenza, il sistema persegue una netta riduzione della barriera tecnica nell'accesso ai dati, semplificando i processi di analisi e permettendo a utenti non esperti di estrarre informazioni complesse in tempo reale, diminuendo la dipendenza da intermediari tecnici.

1.3 Struttura della Tesi

Il presente elaborato è organizzato in capitoli, strutturati secondo un percorso logico che spazia dall'inquadramento teorico fino alla validazione empirica sul campo:

- **Capitolo 2 — Stato dell'arte e Background:** descrive l'evoluzione tecnologica dei Large Language Models e analizza lo stato dell'arte delle metodologie Text-to-SQL e dei database vettoriali. Il capitolo include una rassegna critica dei principali strumenti commerciali attualmente disponibili sul mercato, evidenziandone i limiti strutturali a cui il presente lavoro intende rispondere.
- **Capitolo 3 — Soluzione proposta:** illustra nel dettaglio l'architettura ingegnerizzata a oggetti, descrivendo le responsabilità dello `SchemaOrchestrator` e del `QueryOrchestrator`. Vengono analizzati i flussi logici del back-end FastAPI e l'interfaccia di front-end in React concepita per l'interazione con l'utente finalizzandone i casi d'uso.
- **Capitolo 4 — Test e valutazioni:** espone la metodologia sperimentale adottata per la validazione del framework sui benchmark internazionali Spider e BIRD. In questa sezione vengono analizzati in modo esaustivo i risultati quantitativi ottenuti su molteplici modelli ed ecosistemi di database, arricchendo la trattazione con uno studio inferenziale e statistico delle correlazioni.
- **Capitolo 5 — Conclusione:** riassume i principali traguardi conseguiti in relazione ai requisiti progettuali e delinea le promettenti traiettorie di ricerca future volte all'ottimizzazione del sistema.

2 Stato dell'arte e Background

L'ultimo decennio ha segnato un punto di svolta fondamentale nella storia dell'informatica, caratterizzato da una transizione senza precedenti verso sistemi basati sull'Intelligenza Artificiale (AI). Quella che inizialmente era considerata una disciplina di nicchia è evoluta in un pilastro tecnologico pervasivo, capace di influenzare settori che spaziano dalla medicina alla finanza, fino alla gestione dei dati strutturati.

2.1 Recenti scoperte ed invenzioni

Nell'ultimo periodo abbiamo assistito alla nascita dei Large Language Models (LLM), come la famiglia GPT di OpenAI, Claude di Anthropic e modelli open-source come Llama di Meta. Questi sistemi non si limitano più a una mera elaborazione statistica del linguaggio, ma dimostrano capacità emergenti di ragionamento logico, sintesi e, in particolare, generazione di codice. È proprio in questo alveo di innovazione che si inserisce la capacità di tradurre il linguaggio naturale in linguaggi di programmazione o di interrogazione, aprendo la strada a interfacce uomo-macchina sempre più intuitive e meno dipendenti dalla conoscenza tecnica sintattica.

L'avvento degli LLM ha tuttavia evidenziato alcuni limiti intrinseci legati alla natura statica del loro addestramento e alla tendenza a generare informazioni errate ma sintatticamente plausibili (fenomeno noto come allucinazione). Per superare tali vincoli e connettere i modelli linguistici a basi di conoscenza dinamiche, la ricerca in oggetto si è focalizzata sullo sviluppo di architetture capaci di operare nello spazio della semantica computazionale.

Il cardine di questa evoluzione è rappresentato dagli embeddings, ovvero trasformazioni matematiche che convertono dati discreti, come parole, frasi o immagini, in vettori numerici continui all'interno di uno spazio a dimensionalità ridotta [5]. L'obiettivo fondamentale di queste rappresentazioni è codificare le informazioni semantiche in modo che elementi concettualmente simili risultino matematicamente vicini all'interno dello spazio vettoriale, calcolandone la distanza geometrica [6]. A differenza dei metodi tradizionali basati sulla corrispondenza testuale esatta o sulle parole chiave, gli embeddings catturano relazioni complesse e sfumature del linguaggio, permettendo al sistema di comprendere il contesto e il significato profondo dell'input [7].

Su questa infrastruttura matematica si innesta il paradigma della Retrieval-Augmented Generation (RAG), una metodologia che potenzia i modelli linguistici integrando la loro memoria nativa (memoria parametrica) con un modulo di recupero della conoscenza esterna (memoria non parametrica) [8]. Il processo si articola attraverso la sinergia di due componenti principali [9]:

1. Il modulo di recupero (Retriever): I documenti della base di conoscenza esterna vengono frammentati, convertiti in embeddings e indicizzati. All'atto dell'interrogazione, anche la query dell'utente viene trasformata in un vettore, consentendo al sistema di confrontarla con i documenti indicizzati e identificare le unità informative semanticamente più pertinenti.
2. Il modulo di generazione (Generator): Il contesto recuperato viene inserito nel prompt originale e fornito all'LLM, il quale utilizza queste informazioni aggiornate per produrre una risposta accurata e fattualmente fondata.

Sotto il profilo metodologico, l'approccio RAG offre tre vantaggi cruciali nell'ingegneria dei sistemi basati su AI. In primo luogo, l'ancoraggio a documenti reali mitiga drasticamente il rischio di allucinazioni [8, 9, 10]. In secondo luogo, permette un aggiornamento dinamico della conoscenza, poiché la memoria del sistema può essere espansa o revisionata modificando semplicemente l'indice dei vettori, senza l'oneroso bisogno di riaddestrare il modello [9, 8]. Infine, garantisce un'elevata efficienza computazionale, ottimizzando l'uso della finestra di contesto del modello generativo attraverso la selezione esclusiva dei sottoinsiemi di dati rilevanti [11].

Da un punto di vista infrastrutturale, la gestione e l'interrogazione efficiente di milioni di vettori ad alta dimensionalità richiede tecnologie dedicate: i vector store (o database vettoriali) [7, 10]. Questa tecnologia di archiviazione alimenta la ricerca semantica implementando algoritmi avanzati per la

ricerca dei vicini più prossimi (come l' Approximate Nearest Neighbor), capaci di restituire i vettori più simili in tempi estremamente ridotti [12, 10]. All'interno dei sistemi RAG, il vector store funge da vero e proprio ponte logistico tra l'LLM e i repository di dati; esso memorizza i vettori e gestisce i relativi metadati, facilitando il filtraggio granulare delle informazioni e migliorando la trasparenza del sistema grazie alla tracciabilità e alla citazione delle fonti originali [7, 10].

2.2 Metodologia Text-to-SQL e Integrazione del RAG

La traduzione Text-to-SQL rappresenta un compito cardine del Natural Language Processing (NLP), il cui fine è la conversione automatica di quesiti in linguaggio naturale (NL) in interrogazioni SQL eseguibili da un sistema di gestione di database relazionali (RDBMS) [13, 14]. Questo processo funge da interfaccia intuitiva verso i dati, consentendo l'estrazione di informazioni anche a utenti privi di competenze tecniche [15, 16, 13].

Sotto il profilo computazionale, il problema è modellato come una sfida Sequence-to-Sequence (*Seq2Seq*), in cui l'input unisce il quesito dell'utente e lo schema relazionale del database per generare la stringa SQL corrispondente [16, 14].

Nonostante i progressi dei Large Language Models (LLM), la traduzione accurata sconta criticità legate all'ambiguità semantica e alla scalabilità degli schemi. Nei database reali, la presenza di centinaia di tabelle rende proibitivo l'inserimento dell'intero catalogo nel prompt a causa dei limiti della finestra di contesto e dei relativi costi computazionali [13]. In questo scenario, l'integrazione di tecniche di Retrieval-Augmented Generation (RAG) costituisce una strategia determinante.

L'applicazione del paradigma RAG al Text-to-SQL si realizza principalmente attraverso lo Schema Linking, volto ad allineare i termini della domanda con le entità specifiche del database [15, 11, 17]. Invece di trasmettere l'intero catalogo, un modulo di recupero (retriever) seleziona esclusivamente il sottoinsieme di tabelle e colonne pertinente al quesito [11, 18]. Tale approccio isola uno "schema focalizzato", riducendo il rumore informativo e prevenendo i fenomeni di allucinazione del modello (ovvero il riferimento a identificatori inesistenti) [11, 17].

I principali vantaggi tecnici e metodologici di questa integrazione includono:

- **efficienza dei token e contrazione dei costi:** il filtraggio preventivo dello schema riduce il volume di token inviati all'LLM, abbattendo la latenza e ottimizzando l'impiego di modelli open-weight minori; [11, 18, 17]
- **precisione nello Schema Linking:** l'uso di embedding vettoriali e ricerca semantica cattura relazioni latenti tra i concetti espressi dall'utente e la nomenclatura tecnica, superando i limiti della corrispondenza lessicale; [7, 5]
- **robustezza Cross-Domain:** il framework facilita la generalizzazione in modalità zero-shot su domini mai visti in fase di addestramento, estraendo dinamicamente metadati aggiornati senza richiedere il riaddestramento della rete; [6, 13]
- **strategie di recupero avanzate:** approcci combinati di tipo table-first (identificazione delle tabelle e poi delle colonne) e column-first (selezione degli attributi per risalire alle tabelle) garantiscono un'elevata recall statistica, preservando gli elementi essenziali per la query. [17]

L'integrazione con le tecniche RAG trasforma così il Text-to-SQL da una generazione stocastica "alla cieca" a un processo deterministico guidato dai dati.

Oltre all'ottimizzazione del contesto, la ricerca recente ha introdotto guide metodologiche esplicite per strutturare i processi inferenziali del modello, incrementando l'accuratezza rispetto al prompting diretto.

La tecnica cardine è il Chain-of-Thought (CoT): l'LLM viene istruito a esplicitare il proprio ragionamento step-by-step, scomponendo il problema in passaggi logici intermedi (individuazione delle tabelle, applicazione dei filtri e risoluzione delle giunzioni). [19] Questa scomposizione riduce gli errori logico-strutturali e ottimizza la gestione delle clausole di JOIN complesse.

Infine, il concetto di Self-Correction istituisce un ciclo iterativo a ciclo chiuso in cui il sistema agisce come revisore del proprio output [9, 20]. Qualora l'esecuzione sul DBMS sollevi un'eccezione

sintattica o di runtime, il modello riceve in input il messaggio d'errore nativo e utilizza questo feedback informativo per correggere la query e riavviare l'esecuzione [9]. Tale meccanismo consente di sanare query inizialmente non conformi, innalzando l'accuratezza esecutiva rispetto ai sistemi tradizionali a tentativo unico.

2.3 Approfondimento sui tool presenti in commercio

È interessante notare come il mercato si sia diviso in due segmenti chiari: tool orientati alla Business Intelligence (BI) per data analyst e framework orientati alla produttività dello sviluppatore.

L'attuale ecosistema Text-to-SQL offre una vasta gamma di interfacce web e agenti AI che mirano a democratizzare l'accesso ai dati. L'analisi che segue classifica i tool in base alla loro capacità di integrazione e alla raffinatezza del loro approccio al problema del contesto.

Tra le soluzioni più snelle spicca SQLAI.ai, che fa della flessibilità nella gestione dello schema il suo tratto distintivo; permettendo sia l'input manuale che la connessione diretta, si pone come un supporto versatile per il programmatore, specialmente in contesti dove la sicurezza del dato è prioritaria [21].

Diversa è invece la strategia adottata da Chat2DB, che punta sull'eterogeneità del supporto: la sua architettura permette di interfacciare il sistema con una vasta gamma di modelli linguistici, da Claude a GPT, fino a modelli proprietari via API, rendendolo uno strumento agnostico rispetto al fornitore di IA e adatto ad ambienti enterprise complessi [22].

Sul fronte della personalizzazione e del recupero dell'informazione, Vanna.ai rappresenta lo stato dell'arte dei framework basati su RAG. A differenza di una semplice web-app, Vanna opera come una libreria Python che sfrutta una memoria semantica per apprendere dai metadati e dalle esecuzioni passate, fornendo all'utente una visione trasparente del processo logico [23]. Infine, l'evoluzione verso agenti autonomi è incarnata da AskYourDatabase [24]. Attraverso l'introduzione della cosiddetta 'Think Mode', il sistema esplicita il ragionamento Chain-of-Thought del modello, posizionandosi non solo come generatore SQL, ma come una vera e propria piattaforma di Business Intelligence conversazionale pronta all'integrazione via API.

Questi strumenti mirano a eliminare completamente la barriera del linguaggio SQL, offrendo insight diretti e visualizzazioni. Sia Text2SQL.ai che AI2sql si focalizzano sulla semplicità d'uso per i non tecnici [25, 26]. Mentre Text2SQL.ai eccelle nella spiegazione testuale del codice generato, AI2sql offre strumenti unici per la creazione di schemi da zero (tramite file SQL o descrizioni naturali), risultando utile anche in fase di progettazione del database. BlazeSQL, si posiziona come un assistente di alto livello per team aziendali [27]. Include funzionalità di addestramento specifico sullo schema utente tramite richieste di prova. Questo suggerisce l'uso di tecniche di Few-Shot Prompting per migliorare la precisione nel tempo in base al dominio specifico dell'azienda. Formula Bot e Sequel estendono il concetto di Text-to-SQL all'analisi documentale [28, 29]. Formula Bot, in particolare, permette di unire dati provenienti da database SQL con file CSV e PDF, offrendo una visione olistica dei dati aziendali e la possibilità di scaricare i record analizzati. SQLchat adotta un approccio "Bring Your Own Key": non fornisce un modello interno, ma richiede le chiavi API dell'utente (es. OpenAI) [30]. Questo garantisce all'utente il controllo totale sui costi e sulle versioni del modello utilizzato, pur mantenendo un'interfaccia di chat intuitiva connessa al database.

Dall'analisi emerge che il fattore discriminante tra un tool "giocattolo" e uno professionale è la gestione del contesto (Schema Linking). Mentre i tool più semplici chiedono all'utente di specificare le tabelle (approccio manuale), i sistemi più evoluti come Vanna, AskYourDatabase e BlazeSQL implementano internamente architetture RAG per selezionare autonomamente le informazioni pertinenti, riflettendo le metodologie di ricerca descritte nei capitoli precedenti.

L'analisi dello stato dell'arte evidenzia come il passaggio dal linguaggio naturale al codice SQL non sia più una sfida puramente sintattica, ma un complesso problema di gestione del contesto e del ragionamento logico. Le recenti scoperte nell'ambito dei Large Language Models, unite a metodologie avanzate come il Chain-of-Thought e il Self-Correction, hanno permesso di raggiungere livelli di precisione (Execution Accuracy) inimmaginabili fino a pochi anni fa. Nonostante i progressi compiuti dalle soluzioni commerciali e dai recenti studi accademici, l'analisi evidenzia il permanere di alcune criticità strutturali. In primo luogo, emerge una forte dipendenza dal contesto: l'efficacia di molti strumenti decade sensibilmente in presenza di schemi di database vasti e complessi, qualora non supportati da

meccanismi di recupero (RAG) adeguatamente ottimizzati. Parallelamente, si riscontra una carenza in termini di robustezza operativa: la capacità dei sistemi di gestire autonomamente gli errori di runtime restituiti dal DBMS appare spesso limitata o priva della necessaria trasparenza per l'utente finale. Infine, un limite significativo riguarda la flessibilità di orchestrazione. Raramente i tool attuali offrono una personalizzazione profonda del flusso di lavoro, che permetta di separare nettamente fasi distinte come la gestione dello schema, la generazione della query e la successiva validazione dei risultati, impedendo così un controllo granulare su ogni passaggio del processo. Proprio in questo spazio si inserisce il lavoro di questa tesi. Sulla base delle fondamenta teoriche e tecnologiche analizzate in questo capitolo, nei capitoli successivi verrà presentata un'architettura di Query Orchestrator progettata per massimizzare l'affidabilità della traduzione attraverso un sistema modulare di gestione dello schema e un ciclo iterativo di raffinamento basato sui feedback in tempo reale del database.

3 Soluzione proposta

La soluzione progettata consiste in un sistema di traduzione Text-to-SQL ingegnerizzato per consentire l'interazione con diverse piattaforme di database, prescindendo dalla conoscenza diretta del linguaggio SQL da parte dell'utente. L'architettura si divide in due macro-componenti: un'interfaccia utente (Frontend) sviluppata in React, concepita per garantire un'interazione intuitiva e diretta, e un motore di elaborazione (Backend) implementato in Python e strutturato in moduli software esposti tramite endpoint grazie al framework FastAPI. Il codice sorgente dell'intero sistema, comprensivo dei moduli di backend e frontend, è stato reso disponibile pubblicamente su GitHub [31].

Il cuore logico del backend è governato da due orchestratori principali: lo **SchemaOrchestrator** e il **QueryOrchestrator**. Tali componenti hanno il compito di coordinare i vari moduli del sistema, garantendo l'efficienza dei flussi d'esecuzione.

3.1 Schema Orchestrator

Lo **SchemaOrchestrator** è deputato principalmente all'acquisizione, alla strutturazione e alla persistenza delle informazioni relative allo schema del database relazionale di destinazione. L'obiettivo centrale è generare un'istanza informativa permanente che possa essere condivisa ed efficacemente consultata nel corso delle diverse sessioni utente.

3.1.1 Acquisizione dello schema

Il processo di acquisizione ha inizio verificando l'eventuale presenza di una rappresentazione dello schema precedentemente salvata in formato JSON. Qualora si renda necessaria una nuova estrazione, il sistema prevede due scenari alternativi: l'estrazione diretta tramite un client database (`database_client`) o l'estrazione guidata tramite un Modello Linguistico (LLM) sollecitato da un prompt ingegnerizzato.

Nel primo scenario, il sistema sfrutta il `database_client`, una classe specializzata nell'interazione nativa con il DBMS dell'utente, capace di garantire la massima accuratezza informativa. Il client espone il metodo `extract_schema`, il quale interroga direttamente il dizionario dei dati di sistema (es. la tabella `information_schema`) per recuperare i metadati relativi a tabelle, colonne, tipi di dato e vincoli di integrità referenziale (chiavi esterne). I dati della query di ispezione vengono temporaneamente incapsulati in un oggetto `QuerySession` e gestiti tramite un cursore di connessione. Al fine di garantire la robustezza del software, il ciclo di vita della connessione prevede una gestione delle eccezioni per intercettare fallimenti di runtime o timeout d'esecuzione. Se l'operazione ha successo (`SUCCESS`), le righe estratte vengono modellate in una struttura dati custom denominata `Records`, per poi essere formattate e mappate nelle corrispondenti posizioni del file JSON dello schema.

Nel **secondo scenario**, che si attiva qualora l'utente fornisca file DDL o descrizioni testuali del database in linguaggio naturale, l'interazione con i modelli generativi è governata da una gerarchia di classi fondata sulla classe astratta `BaseLLM`. Quest'ultima definisce l'interfaccia comune per tutti i connettori LLM del sistema, esponendo il metodo astratto `generate(prompt, kwargs)` e incapsulando

proprietà condivise come il meccanismo di gestione del timeout `TIMEOUT_PER_REQUEST`, impostato di default a dieci minuti.

L'estensione di tale interfaccia è implementata tramite componenti specializzate per i singoli provider:

- **AzureLLM:** si interfaccia nativamente con i servizi di Azure OpenAI sfruttando un loader interno (`AzureLoader`) per il recupero sicuro delle credenziali e degli endpoint. Il metodo `generate` esegue la chiamata verso le API di chat completion e incapsula l'operazione all'interno di un costrutto `try-except`, sollevando un'eccezione esplicita di `TimeoutError` qualora l'operazione superi la finestra temporale prefissata.
- **OpenWebUILLM:** estende `BaseLLM` introducendo una flessibilità strutturale basata sull'attributo `api_type`, che discrimina a runtime tra interfacce basate su chat (`chat`) e interfacce basate su completamento del testo (`completion`). Tale biforcazione logica viene gestita istanziando dinamicamente il loader configurativo appropriato (`ChatLoader` o `CompLoader`) per l'estrazione dei parametri di connessione e delle chiavi di autenticazione.

La gestione del ciclo di vita della richiesta in `OpenWebUILLM` si appoggia sempre sul metodo `generate`, il quale inoltra una richiesta HTTP POST sincrona verso l'endpoint configurato utilizzando la libreria `requests`. Per far fronte alla natura duale dell'API, la classe implementa due metodi privati dedicati: `_dynamic_payload`, deputato alla generazione del corpo della richiesta formattato con i parametri posizionali o con la cronologia dei messaggi, e `_dynamic_response`, incaricato di eseguire il parsing selettivo del dizionario JSON restituito dal server.

Un elemento di rilievo ingegneristico all'interno di `_dynamic_response` è rappresentato dal meccanismo preventivo di intercettazione degli errori di contesto. Mediante un'analisi lessicale della risposta del server, il sistema verifica se l'anomalia sia riconducibile al superamento dei limiti fisici della finestra di contesto del modello, mappando token critici quali `n_ctx` o `context length`. In caso positivo, viene sollevata un'eccezione mirata di `ValueError`, impedendo la propagazione di fallimenti generici e fornendo un feedback strutturato ai moduli superiori dell'applicazione. Infine, la conformità con la politica di tolleranza ai guasti dell'architettura è preservata catturando esplicitamente le eccezioni di tipo `requests.exceptions.Timeout`, che vengono uniformate e rilanciate sotto forma di `TimeoutError`.

In questo caso, la classe `PromptBuilder` genera dinamicamente un prompt (`schema_generation_prompt 3.1.1`) inserendo l'input dell'utente in un contesto predefinito. Il prompt definisce il ruolo del modello (esperto di basi di dati) e impone rigidi vincoli strutturali, richiedendo espressamente un output in formato JSON valido, privo di commenti o testo conversazionale generico.

Al fine di validare e rendere persistente il risultato ottenuto da entrambi gli scenari, viene invocato il metodo privato `_process_response`. Questo delega il parsing sintattico alla componente `Schema` mediante il metodo `parse_response`. Per mitigare la variabilità e le imperfezioni tipiche degli output dei modelli linguistici, il meccanismo di parsing effettua in sequenza quattro strategie di recupero euristico:

- **Parsing JSON diretto:** tentativo di deserializzazione standard del testo.
- **Isolamento tramite parentesi graffe:** estrazione del testo compreso tra la prima e l'ultima parentesi graffa, supportato da una pulizia automatica delle virgole terminali (trailing commas).
- **Markdown Fenced Blocks:** estrazione del codice circoscritto dai tripli backtick (`"`"`).
- **Widest Candidate:** ricerca iterativa del frammento JSON valido più esteso presente nel corpo del testo.

Una volta convalidata la struttura, il file JSON viene salvato localmente. Contestualmente, lo `SchemaOrchestrator` invoca il metodo `add_schema` dello `SchemaStore` per indicizzare le tabelle appena estratte all'interno del sistema RAG (Retrieval-Augmented Generation). Il metodo `to_documents` converte i metadati di ciascuna tabella in documenti compatibili con il vector store utilizzato. Durante questa fase, eventuali documenti preesistenti riferiti allo stesso database vengono preventivamente rimossi dallo store per evitare conflitti di versione e garantire l'allineamento dei dati in caso di aggiornamento.

3.1.2 Aggiornamento dello schema

Qualora l'utente desideri arricchire o modificare la conoscenza dello schema, il sistema consente l'aggiornamento incrementale attraverso il metodo `update_current_schema`. Previa verifica dell'esistenza del file JSON di base, viene eseguita un'analisi euristica sull'input dell'utente mediante il metodo `classify_update`. Questo analizza la presenza di specifiche parole chiave per determinare la natura della modifica. Se l'aggiornamento non altera l'architettura dei dati ma aggiunge dettagli descrittivi, il testo viene semplicemente archiviato nel campo `semantic_notes` della classe `Schema`. Se l'input implica variazioni su tabelle o colonne, viene strutturato un nuovo prompt di aggiornamento (`schema_update_prompt` 3.1.2) tramite il `PromptBuilder`. Questo prompt include lo schema corrente e le istruzioni per integrare le modifiche. La risposta dell'LLM viene infine processata nuovamente dal metodo `_process_response`, garantendo la medesima catena di validazione e persistenza utilizzata in fase di prima acquisizione.

3.2 Query Orchestrator

Il `QueryOrchestrator` è il componente core deputato alla traduzione semantica delle richieste formulate dall'utente in linguaggio naturale in istruzioni SQL eseguibili. Ereditando i metadati strutturali precedentemente consolidati dallo `SchemaOrchestrator`, questo modulo implementa un ciclo iterativo di generazione, valutazione e raffinamento che consente al sistema di correggere i propri output analizzando i fallimenti d'esecuzione.

All'atto dell'inizializzazione, l'orchestratore riceve come parametri l'identificativo del database, un'istanza dello `SchemaStore`, il modello linguistico selezionato e, opzionalmente, il modulo `data_base_client`. La presenza o l'assenza di quest'ultimo determina una biforcazione logica nel flusso elaborativo, discriminando la possibilità di effettuare o meno una validazione dinamica e sperimentale sul DBMS dell'utente. In fase di istanziazione, l'orchestratore si avvale del design pattern Factory tramite la classe `LLMFactory`, il cui metodo statico `create` esamina il dizionario di configurazione per isolare le specifiche del provider (come chiavi crittografiche ed endpoint di connessione) e restituire il corrispondente wrapper specializzato (`BaseLLM`).

3.2.1 Generazione e Raffinamento Iterativo della Query

Il processo principale è governato dal metodo `generation`, il cui obiettivo consiste nell'esaminare la richiesta e restituire un oggetto `QuerySession` contenente il codice SQL finale e i metadati associati al ciclo di vita della sessione. Tramite il metodo privato `_init_generation_context`, il sistema interroga lo `SchemaStore` per ricavare la porzione di schema rilevante (`schema_context`). Al fine di ottimizzare la finestra di contesto dell'LLM ed evitare l'inclusione di tabelle superflue, lo store applica un'euristica basata sulla stima della complessità dell'input. Analizzando la presenza di specifici costrutti linguistici riconducibili a operazioni di aggregazione o di join, il sistema espande dinamicamente il parametro di recupero k partendo da un valore basale ($k = 3$) fino a un massimo di 7 tabelle. I documenti così selezionati dal vector store vengono de-duplicati e formattati. Qualora sia disponibile una connessione attiva tramite il `database_client`, il contesto viene ulteriormente integrato mediante due sorgenti informative fondamentali:

- **Analisi dei Fallimenti Pregressi (`QueryStore`):** Il sistema interroga un secondo database vettoriale (`QueryStore`) contenente lo storico delle query fallite. Tramite una ricerca per similarità semantica combinata con una funzione di decadimento temporale (che predilige gli errori più recenti), vengono estratti fino a 3 documenti associati a tentativi precedentemente contrassegnati da uno stato di errore (es. `SYNTAX_ERROR` o `RUNTIME_ERROR`). Tali elementi vengono passati come vincolo negativo all'LLM per impedirgli di ripetere il medesimo errore logico.
- **Suggerimenti di Join (`join_hints`):** Sfruttando le capacità introspettive del `database_client`, l'orchestratore interroga i metadati di sistema (quali `information_schema.KEY_COLUMN_USAGE`) per mappare le relazioni di chiave esterna attive tra le tabelle coinvolte nella richiesta. L'output, tradotto in una lista di vincoli strutturali, guida esplicitamente il modello nella stesura delle clausole di JOIN.

Completata la fase di recupero del contesto, il sistema avvia un ciclo iterativo di ottimizzazione governato da soglie rigide volte a prevenire loop infiniti o un consumo eccessivo di token. Il ciclo si interrompe non appena la query viene validata con successo o, in alternativa, al raggiungimento di 5 errori di runtime consecutivi (`runtime_error_limit`) o di 3 riscontri semantici negativi (`evaluation_attempts_limit`).

La formulazione del codice SQL per ogni singolo tentativo è demandata al metodo `_generate_sql_attempt`, il quale invoca la classe `PromptBuilder` generando il prompt ingegnerizzato `query_generation_prompt` A.3. Il testo generato organizza rigorosamente la richiesta dell'utente, il dialetto SQL di destinazione, lo schema estratto, i suggerimenti sulle chiavi esterne, lo storico dei fallimenti e, infine, un piano logico strutturato che impone al modello di determinare internamente le tabelle necessarie e le relazioni d'aggiornamento prima di emettere esclusivamente il codice SQL finale.

Alla ricezione della stringa testuale prodotta dal server dell'LLM, viene avviata una routine di sanificazione sintattica (`clean_sql_from_llm`) all'interno dell'oggetto `QuerySession`. Questa operazione è necessaria per eliminare i frequenti artefatti dei modelli linguistici, rimuovendo blocchi di codice Markdown, espressioni di controllo spurie (es. `<0x0A>`) e troncando qualsiasi testo esplicativo successivo all'ultimo punto e virgola.

La fase critica di verifica prevede due scenari determinati dall'architettura d'istanziamento:

- **Assenza di Client Database:** La valutazione si limita alla verifica formale e sintattica della stringa mediante un parser astratto (`sqlglot`), il quale accerta l'assenza di violazioni grammaticali nella struttura SQL.
- **Presenza di Client Database:** Viene invocato il metodo `evaluation`, che esegue direttamente la query sul database bersaglio impostando un timeout di sicurezza sul cursore d'esecuzione. In caso di successo, il risultato viene sottoposto a una seconda valutazione semantica demandata a un modello valutatore, deputato a verificare se le righe ottenute rispondano fedelmente alla domanda iniziale. In caso di fallimento ripetuto a runtime, il valutatore genera un feedback esplicativo sulle cause del problema che verrà iniettato nell'iterazione successiva.

Al termine di ciascun tentativo, il metodo `evaluate` classifica formalmente la sessione valorizzando lo stato finale (`QueryStatus`), la categoria dell'anomalia (`ErrorType`) e il dominio di competenza dell'errore (`KnowledgeScope`), isolando i problemi puramente sintattici da quelli strutturali (es. clausole `GROUP BY` mancanti) o strettamente legati ai vincoli dello schema.

Infine, ogni transazione viene registrata all'interno del `QueryStore` corredata da un set esteso di metadati (timestamp, numero di tentativi, categorie d'errore ed esito del DBMS). Tale storicizzazione assicura l'apprendimento continuo del sistema, permettendo all'orchestratore di capitalizzare l'esperienza accumulata e di restituire, alla chiusura del ciclo, la sessione d'ispezione più efficiente ed accurata.

3.2.2 Valutazione della query

Il metodo `evaluation` presiede alla verifica formale, esecutiva e semantica della query SQL generata. Questo processo di validazione approfondita è subordinato alla disponibilità del `database_client`, elemento indispensabile per interagire direttamente con l'ambiente DBMS di destinazione e raccogliere i riscontri empirici d'esecuzione.

All'avvio del modulo, la query candidata viene eseguita sul database dell'utente. L'esito di tale operazione determina l'attivazione di due circuiti di feedback distinti, gestiti in sinergia con un modello linguistico indipendente configurato con il ruolo di valutatore (LLM Evaluator).

Qualora l'esecuzione sul database fallisca a causa di un'eccezione del DBMS, il sistema differisce l'intervento dell'LLM valutatore e incrementa immediatamente il contatore interno degli errori di runtime (`runtime_errors`). Se l'anomalia persiste nelle iterazioni successive, viene attivata una contromisura deterministica: la classe `PromptBuilder` formula un prompt di spiegazione (`explanation_prompt` A.4) che isola la richiesta iniziale, il codice SQL errato e il tracciato dell'eccezione restituito dal database. L'LLM Evaluator analizza questo contesto per generare una diagnosi mirata del fallimento. Tale risposta viene strutturata come un vincolo correttivo e iniettata dinamicamente nel prompt di

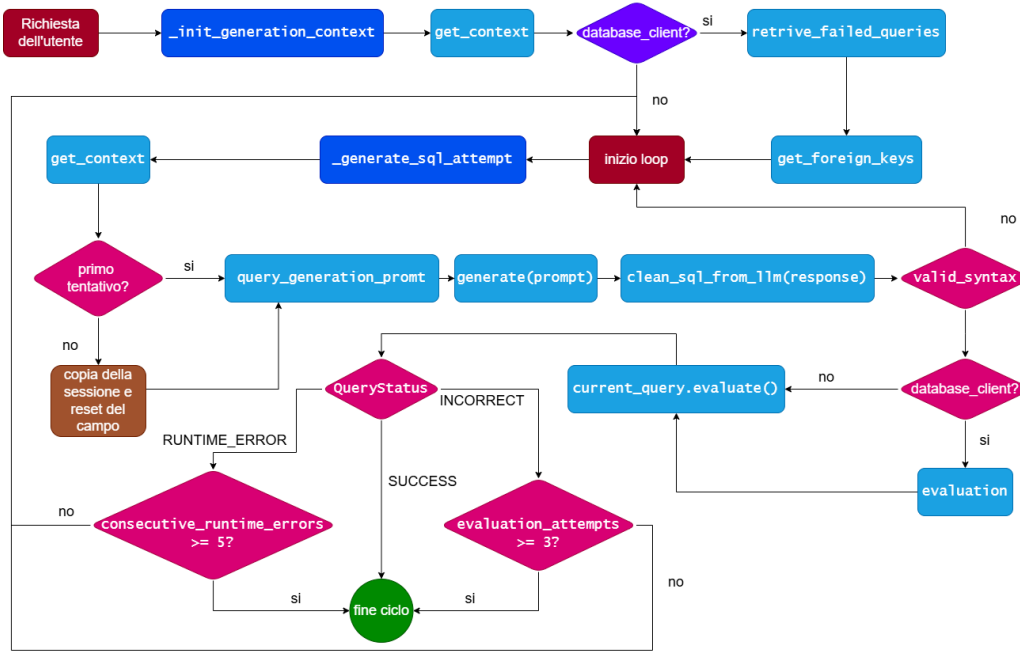


Figura 3.1: Flusso iterativo della generazione del codice SQL.

generazione del tentativo successivo, guidando il modello di generazione verso la risoluzione del bug sintattico o strutturale.

In caso di esecuzione riuscita della query, il sistema deve accertare che i dati estratti rispondano fedelmente all'intento originario dell'utente, scongiurando il rischio di query sintatticamente valide ma logicamente errate. A tal fine, viene strutturato un prompt di valutazione (**evaluation_prompt** A.5) che include la richiesta in linguaggio naturale, il codice SQL prodotto e un campione rappresentativo dei dati estratti, limitato ai primi venti record (**Records.get_preview**) per ottimizzare l'occupazione della finestra di contesto del modello. L'LLM Evaluator esamina la coerenza tra la struttura dei dati ottenuti e la semantica della domanda. Qualora il verdetto sia positivo, lo stato della sessione (**QueryStatus**) viene consolidato come **SUCCESS**, interrompendo l'algoritmo iterativo e restituendo la query validata.

Al termine di ciascuna iterazione, le metriche di classificazione e i verdetti emessi dall'Evaluator vengono storicizzati all'interno dell'oggetto **QuerySession**, alimentando la base di conoscenza del **QueryStore** per i successivi cicli di recupero del contesto.

3.3 Interfaccia

A completamento dell'architettura di backend descritta, è stata sviluppata un'interfaccia utente dedicata, progettata per astrarre la complessità dei processi di orchestrazione e fornire un punto di accesso semplificato alle funzionalità di generazione e valutazione SQL. L'applicazione frontend è strutturata in diverse viste logiche, accessibili tramite una barra di navigazione persistente che include le sezioni Home, Database e Generate.

La sezione dedicata alla gestione dei database rappresenta il punto di innesco per lo **SchemaOrchestrator**. L'obiettivo di questa interfaccia è permettere all'utente di definire il contesto relazionale su cui opereranno i modelli di linguaggio. Il sistema offre due modalità operative distinte. L'utente può caricare file di definizione dati (DDL) o inserire manualmente istruzioni **CREATE TABLE**. In questa modalità, l'interfaccia delega al sistema l'inferenza della struttura tramite LLM per la produzione dello schema JSON. Invece sfruttando il **MySQLClient**, l'utente può stabilire una connessione attiva con un server MySQL. L'interfaccia presenta dinamicamente una lista dei database accessibili, permettendo la selezione puntuale del target.

In entrambi i casi, l'utente è tenuto a specificare degli identificativi mnemonici (il nome logico all'interno dell'applicazione e l'alias da utilizzare nelle query) per garantire la corretta risoluzione

dei riferimenti durante la generazione del prompt. Una volta confermati i dati, lo schema viene normalizzato, indicizzato nel **SchemaStore** (Vector Database) e iniettato nella sessione attiva.

La pagina di generazione costituisce l'interfaccia di controllo del **QueryOrchestrator**. Attraverso questa vista, l'utente può configurare i parametri della sessione di interrogazione:

1. **Selezione del Contesto:** Scelta del database tra quelli precedentemente configurati nella sessione.
2. **Configurazione del Modello:** Selezione del Large Language Model (LLM) tramite la **LLMFactory**, permettendo di testare la generazione su provider differenti (es. Azure o OpenWebUI).
3. **Input in Linguaggio Naturale:** Un'area di testo dedicata accoglie la richiesta dell'utente.

Al termine del processo di generazione, l'interfaccia non si limita a mostrare il codice SQL prodotto, ma fornisce un feedback immediato sull'efficacia della query. Se il database è stato collegato tramite connessione diretta, il sistema esegue automaticamente la query e presenta un'anteprima dei record estratti. Questo approccio permette all'utente di validare istantaneamente non solo la correttezza sintattica, ma anche la pertinenza semantica dei risultati rispetto alla domanda originale.

Dalla Dashboard principale (Home), l'utente può indirizzare il workflow verso due direttrici funzionali:

- **Generate Query:** Focalizzata sulla creazione ex-novo di codice SQL a partire da requisiti testuali.
- **Evaluate Query:** Dedicata alla revisione critica di query esistenti, dove il sistema agisce come supervisore esperto per identificare errori logici o strutturali, sfruttando la logica di cross-check tra richiesta originale e metadati dello schema.

L'anteprima dell'interfaccia è presente nell'allegato

4 Test e valutazioni

In questo capitolo viene illustrata la metodologia adottata per testare e validare il funzionamento del sistema di traduzione Text-to-SQL progettato. Al fine di garantire l'oggettività e la coerenza dei test prestazionali, la fase di sperimentazione ha previsto l'impiego di due dei dataset di riferimento nel panorama internazionale per la generazione di codice SQL: Spider e BIRD (Big Bench for Large-scale Database Grounded Text-to-SQL Evaluation).

Ciascun dataset fornisce un corpus eterogeneo di interrogazioni in linguaggio naturale correlate alle rispettive query di riferimento (denominate gold queries), oltre agli schemi logici dei database in formato JSON e a uno script di validazione automatico. Tali script consentono di confrontare l'output della query predetta con quello della query target, definendo un benchmark rigoroso per la valutazione delle performance.

Il primo dataset, Spider, dispone di venti database distinti, reperiti attraverso risorse accademiche, tutorial dedicati e portali online specializzati, tra cui DatabaseAnswers e WikiSQL [32]. La nomenclatura di tabelle e colonne è stata preventivamente normalizzata (ad esempio, trasmutando l'identificativo `stu_id` in `student_id`) per assicurare che i metadati risultassero auto-esplicativi; tale accorgimento permette ai modelli linguistici di concentrarsi sul parsing semantico anziché sulla decifrazione di abbreviazioni criptiche [32]. Le 1034 richieste di input sono state generate manualmente da studenti di informatica esperti in linguaggio SQL, escludendo l'uso di template automatici al fine di preservare la naturalezza e la variabilità del linguaggio umano [32]. Le domande di Spider coprono i principali costrutti SQL all'interno di ciascun pool di database e sono state concepite per essere chiare ed univoche [32]. Sono state deliberatamente escluse richieste che imponessero il ricorso a conoscenze esterne al database o inferenze basate sul senso comune [32]. Per garantire la più alta qualità dell'input possibile, il dataset è stato inoltre sottoposto a molteplici fasi di revisione e validazione incrociata [32].

Il benchmark BIRD si discosta parzialmente dall’approccio precedente, attingendo a scenari applicativi reali provenienti da piattaforme come Kaggle o repository pubbliche di GitHub, andando a comporre un set di dieci database complessi [33]. Le 1534 richieste sono state generate tramite un processo di crowdsourcing rigoroso strutturato secondo precisi criteri di complessità logica, introducendo per la prima volta quesiti che richiedono capacità di External Knowledge Reasoning (ragionamento basato su conoscenza esterna) [33]. Ogni singola domanda è stata preventivamente testata per garantire che la corrispondente query SQL di riferimento non restituisse un set di risultati nullo [33].

Entrambi i framework offrono metriche dedicate per verificare l’accuratezza dei risultati. Per quanto riguarda Spider, l’obiettivo primario risiede nel testare la capacità di un modello di generalizzare su nuovi schemi relazionali e domini mai visti in fase di addestramento (cross-domain evaluation), focalizzandosi sulla complessità sintattica della query generata [32]. La valutazione scompone l’interrogazione nei suoi componenti principali (**SELECT**, **WHERE**, **GROUP BY**, ecc.), analizzandoli come insiemi non ordinati di sotto-componenti al fine di gestire l’assenza di vincoli sequenziali rigidi nel linguaggio SQL. Il giudizio algoritmico si articola in tre metriche fondamentali: Component Matching, Exact Matching e Hardness Criteria [32].

Il benchmark BIRD nasce invece con l’intento di superare i limiti strutturali di Spider, introducendo database su larga scala (pari a 33,4 GB complessivi) caratterizzati da dati non normalizzati e affetti da rumore di fondo, simulando fedelmente i contesti industriali reali [33]. Di conseguenza, i criteri di valutazione si evolvono verso una validazione prettamente esecutiva, articolandosi in: Execution Accuracy (EX), Valid Efficiency Score (VES) ed External Knowledge Reasoning[33].

La suite di test è stata eseguita su una selezione di Modelli Linguistici ad Ampio Spettro (LLM) di ultima generazione, afferenti ai principali provider del settore e ottimizzati per la generazione di codice: *GPT-4o*, *GPT-5-mini*, *Codestral (22B)*, *CodeLlama (34B)*, *SQLCoder (34B)* e *Qwen3-Coder-Next*.

4.1 Architettura del Sistema di Testing

Per condurre la campagna di test in modo efficiente, mitigando i tempi morti legati alle latenze delle chiamate API esterne, è stata progettata un’architettura software multithreading. Il sistema distribuisce il carico computazionale istanziando un thread concorrente per ciascun modello LLM configurato (*Generator Thread*). Questo ecosistema viene coordinato da un ulteriore thread concorrente dedicato al monitoraggio (*Printer Thread*). Tale parallelizzazione consente di sottoporre simultaneamente lo stesso set di quesiti a tutti i modelli, ottimizzando i tempi complessivi di esecuzione del benchmark.

Il processo d’ispezione si articola attraverso una sequenza deterministica di fasi. All’avvio, il sistema acquisisce i parametri operativi da riga di comando o, in subordine, tramite una procedura guidata interattiva. L’utente definisce il dataset di riferimento (Spider o BIRD), il database bersaglio e la modalità operativa del test, discriminando tra l’approccio puramente testuale (**text**) e quello basato sulla connessione diretta al DBMS (**db_conn**). L’architettura applica il principio della programmazione orientata agli oggetti mediante la classe astratta **BaseDataset**. Questa definisce l’interfaccia e i metodi standard che le sottoclassi concrete **SpiderDataset** e **BirdDataset** sono obbligate a implementare. All’atto dell’istanziamento, la sottoclasse selezionata popola le proprie strutture dati caricando i file descrittivi del benchmark: il file **dev.json** (contenente i prompt utente), il file **tables.json** (contenente le definizioni degli schemi) e i puntatori ai moduli di computazione delle metriche ufficiali. Il sistema esegue una routine di sanificazione ambientale, azzerando le directory temporanee (**tmp**) e predisponendo la struttura delle cartelle deputate ad accogliere i log e i report intermedi. Contestualmente, viene inizializzato l’oggetto **Schema** e agganciato al corrispondente **SchemaStore**. Un elemento critico del design software riguarda la persistenza dei fallimenti. Poiché tutti i *Generator Threads* accedono contemporaneamente alla risorsa condivisa **QueryStore** per registrare e consultare le query errate, si rende necessario prevenire fenomeni di race condition. Il problema è stato risolto implementando la classe **ThreadSafeQueryStore**, una derivazione di **QueryStore** che incapsula le primitive di scrittura e lettura all’interno di un meccanismo di mutua esclusione (Thread Lock).

Una volta avviati, i *Generator Threads* istanziano dinamicamente un’apposita componente **QueryOrchestrator**, configurata con i parametri del modello e del client specifici del thread. Successivamente, il thread itera sul set di quesiti in linguaggio naturale estratti dal dataset, invocando il metodo **generation**.

I risultati d’inferenza vengono incapsulati in un oggetto `QuerySession`. Qualora l’orchestratore concluda l’elaborazione con uno stato di successo (`SUCCESS`), la sessione viene inoltrata al modulo di valutazione del dataset per subirne il giudizio assoluto. Al fine di garantire l’affidabilità dei log, il thread implementa una gestione delle eccezioni di runtime: il sollevamento di un `TimeoutError` (dovuto a saturazione del canale o latenza del provider) comporta la classificazione esplicita dello stato come `TIMEOUT_ERROR`, mentre le anomalie impreviste vengono mappate sotto la categoria generale `RUNTIME_ERROR`. In entrambi i casi, il sistema genera un oggetto `RequestResult` contrassegnato da un flag di fallimento, impedendo il blocco dei thread paralleli.

Il *Printer Thread* opera in regime di ascolto passivo su una coda sincronizzata (`result_queue`). Man mano che i thread di generazione depositano i rispettivi oggetti `RequestResult`, il coordinatore aggiorna una matrice di stato interna (`results_by_index`) e notifica l’avanzamento su terminale standard tramite la funzione `print_model_progress`. Al completamento di ciascun quesito da parte di tutti i modelli, il sistema scrive un file testuale intermedio, minimizzando il rischio di perdita dei dati in caso di interruzione accidentale. Al termine della sessione globale, il thread consolida la matrice dei risultati generando il report conclusivo aggregato per metriche temporali, accuratezza ed efficacia dei metodi di raffinamento.

La verifica dell’accuratezza delle query SQL prodotte rappresenta un compito complesso, poiché una medesima richiesta semantica può essere espressa attraverso costrutti sintattici differenti ma logicamente equivalenti. I validatori standard mostrano spesso una rigidità eccessiva, portando alla luce numerosi “falsi negativi”: query considerate errate a causa di discrepanze formali marginali, come l’ordine delle colonne o la loro rinominazione. Per ovviare a tali limiti, è stato architettato un framework di valutazione stratificato su tre livelli sequenziali.

Il primo livello si interfaccia direttamente con l’infrastruttura di verifica ufficiale fornita a corredo dei dataset mediante il metodo `dataset_evaluation`. Sia per Spider che per BIRD, l’ispezione avviene invocando un sottoprocesso isolato di sistema che esegue la query predetta sull’ambiente database nativo. L’output del sottoprocesso viene catturato e strutturato in un oggetto `OfficialEvalReport`. La metrica cardinale estratta è la *Execution Accuracy* (`execution_accuracy`), un indicatore numerico espresso come float (1.0 in caso di perfetta corrispondenza tra i record estratti dalla query predetta e quelli della *gold query*, 0.0 in caso di disallineamento o errore strutturale).

I validatori ufficiali dei dataset mostrano un’elevata suscettibilità a variazioni non sostanziali, quali l’ordinamento delle righe o la nomenclatura interna delle colonne, generando talvolta fenomeni di falso negativo. Il secondo livello d’ispezione interviene per mitigare tale rigidità effettuando un confronto diretto e controllato dei set di dati risultanti. Il sistema esegue una routine di normalizzazione dei tipi di dato (uniformando formati di stringhe, date e floating-point) e analizza i risultati secondo regole euristiche:

- **Exact Match:** Identità assoluta dei set di record estratti, che convalida la query restituendo il valore `ComparisonResult.EXACT_MATCH`.
- **Row Count Mismatch:** Discrepanza quantitativa nel numero di tuple restituite (`ComparisonResult.ROW_COUNT_MISMATCH`).
- **Sottoinsieme di Colonne:** Verifica algebrica volta ad accertare se l’anomalia sia limitata all’inclusione o proiezione di colonne spurie, mantenendo l’integrità del nucleo informativo richiesto.

Qualora la query fallisca le verifiche deterministiche precedenti, il framework demanda il giudizio definitivo a un modello linguistico d’alto livello configurato come arbitro semantico (`_run_llm_judge`). Il giudice finale altro non è che un LLM scelto appositamente per le sue notevoli conoscenze linguistiche e la sua capacità di ragionamento logico, nello specifico si tratta di Qwen3-Coder-Next. Sfruttando la classe `PromptBuilder`, viene formulato un prompt che espone la *gold query* con il rispettivo set di dati atteso, e la query generata con il suo specifico output esecutivo A.7. Il modello ha il compito di trascendere le differenze sintattiche superficiali per valutare se la logica della query predetta rispetti l’intento informativo dell’utente. L’output dell’oracolo viene strutturato in un dizionario standardizzato contenente un verdetto (`verdict: correct/incorrect`) e una stringa motivazionale dettagliata (`reason`) in caso di esito negativo.

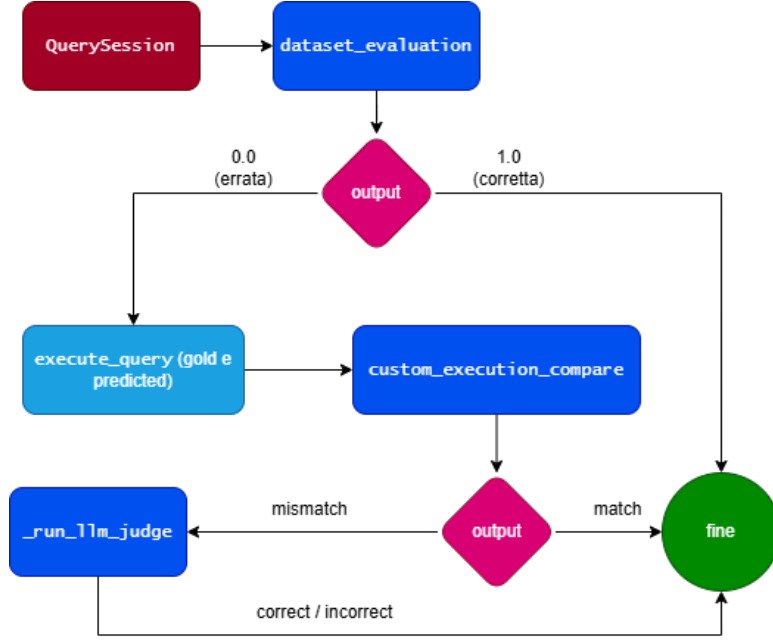


Figura 4.1: Flusso di valutazione della query SQL con il dataset.

L'intero ciclo si conclude con la restituzione di un oggetto `EvaluationResult`, il quale mappa in modo trasparente lo storico dei tentativi, le query coinvolte e i dettagli analitici emessi dai tre livelli di controllo, offrendo una base dati robusta per l'analisi comparativa delle prestazioni dei singoli modelli.

4.2 Fase di analisi

Al fine di garantire il rigore scientifico e la perfetta riproducibilità della fase di valutazione sperimentale, è stato sviluppato un modulo software dedicato alla gestione, standardizzazione ed elaborazione statistica dei tracciati di esecuzione prodotti dai *Large Language Models*. Il componente, implementato nello script `summarize_models.py`, agisce come un motore di aggregazione cross-model, operando a partire dai log strutturati in formato JSON memorizzati al termine delle sessioni di test per ciascun database del benchmark.

Il modulo implementa un'architettura di calcolo che mappa i risultati empirici secondo le due distinte modalità operative di interazione con lo schema relazionale introdotte precedentemente: la modalità connessa (`db_conn`), basata sull'ispezione dinamica e sull'estrazione nativa dei metadati dal DBMS, e la modalità puramente testuale (`text`), vincolata alla fornitura statica delle istruzioni DDL nel prompt dell'LLM.

L'analisi quantitativa si articola su una matrice di vettori e variabili strutturate per valutare la resilienza, l'efficienza e l'accuratezza semantica dell'intero sistema. I parametri cardinali e le metriche integrate all'interno del report di sintesi sono così definiti:

- **Dataset e Database:** identificativi univoci del dominio applicativo e dell'istanza relazionale sottoposta a benchmarking.
- **Metadati Strutturali** (N_t , N_c): rispettivamente, il numero di entità relazionali (tabelle, N_t) e il numero complessivo di attributi (colonne, N_c) che compongono lo schema target.
- **Densità Relazionale** (D_r): rapporto metrico espresso come $D_r = N_c/N_t$, indicativo del livello medio di capienza informativa e di ramificazione delle tabelle.
- **Requests:** il volume campionario totale di query espresse in linguaggio naturale sottoposte al sistema per lo specifico schema.

- **Triplet Score:** rappresentazione vettoriale della distribuzione della complessità delle query, ripartita in tre macro-classi discrete sulla base di una soglia empirica determinata dal numero di costrutti logici rilevati:

$$\text{Complessità} = \begin{cases} \text{Bassa (Low)}, & \text{se Score} \leq 3 \\ \text{Media (Medium)}, & \text{se } 3 < \text{Score} \leq 6 \\ \text{Alta (High)}, & \text{se Score} > 6 \end{cases}$$

- **Avg Query Complexity (Q):** media aritmetica del vettore dei punteggi di complessità intrinseca associati a ciascuna richiesta utente all'interno del database.
- **Complexity Score (C_{db}):** indice normalizzato della complessità architetturale globale del database, derivato combinando la complessità sintattica delle richieste con la densità dello schema relazionale.
- **Success Rate:** frequenza relativa dei successi del modello (calcolata distintamente per le modalità `db_conn` e `text`), espressa come rapporto percentuale tra le query validate come corrette dall'infrastruttura di controllo e i tentativi totali eseguiti nell'ambiente operativo.

Per ponderare in modo oggettivo le performance dei singoli modelli in base alla difficoltà intrinseca del contesto relazionale, il modulo implementa una funzione di normalizzazione della complessità del database. Definita $S = N_c/N_t$ la densità dello schema corrente e $S_{\max}$ il valore massimo registrato nell'intero benchmark, il componente calcola il punteggio finale di complessità del database (C_{db}) mediante la seguente equazione lineare:

$$C_{db} = Q \cdot \left(1 + 0.25 \cdot \frac{S}{S_{\max}} \right)$$

Il modello matematico applicato assicura che l'architettura dello schema relazionale agisca come fattore moltiplicativo di penalizzazione (fino a un massimo del 25%) sulla complessità media delle query (Q), riflettendo l'effettivo overhead cognitivo imposto ai modelli LLM dalla presenza di schemi estesi e densamente popolati.

Contestualmente, per l'analisi approfondita dei casi di insuccesso, lo script esegue una categorizzazione deterministica degli output non conformi, isolando tre macro-classi di errore:

1. **Syntax Error:** inabilità del modello a generare un costrutto SQL sintatticamente valido per il dialetto di riferimento, intercettato formalmente in fase di parsing o validazione astratta.
2. **Runtime Error:** eccezioni sollevate direttamente in fase di esecuzione sul DBMS (quali riferimenti ad attributi o tabelle inesistenti e violazioni dei vincoli di integrità referenziale).
3. **Incorrect (Semantic Error):** query eseguite correttamente dal motore di database senza sollevare eccezioni, ma caratterizzate da un disallineamento semantico (esito logico e tuple estratte divergenti) rispetto alla ground truth di valutazione.

Infine, per comprendere i fattori esogeni ed endogeni che governano il decadimento prestazionale o la robustezza dei modelli, lo script esegue un'analisi inferenziale calcolando i coefficienti di correlazione lineare di Pearson (ρ) e di correlazione per ranghi di Spearman (r_s). L'algoritmo allinea i vettori delle metriche continue (es. numero di tentativi effettuati, complessità normalizzata della query) con i vettori dicotomici associati all'esito della sessione ($Y \in \{0, 1\}$, dove 1 indica il successo e 0 il fallimento).

Il *Coefficiente di Pearson* (ρ) viene impiegato per misurare l'intensità di una relazione lineare tra la metrica indipendente e la probabilità di successo dell'output, mentre il *Coefficiente di Spearman* (r_s) viene applicato per catturare relazioni monotoniche non strettamente lineari, configurandosi come un indicatore robusto in presenza di distribuzioni non parametriche o fortemente sbilanciate.

Ciascun coefficiente viene validato mediante il calcolo del rispettivo valore di significatività (*p-value*). Il sistema rigetta l'ipotesi nulla (H_0 : assenza di correlazione statistica) impostando un livello di significatività alfa standard $\alpha = 0.05$. Qualora uno dei vettori presenti con varianza nulla, tipica presenza di una costanza assoluta dei successi o dei fallimenti per un determinato modello, l'algoritmo contrassegna il coefficiente come *Non Rilevabile* (ND), prevenendo anomalie computazionali e terminazioni anomale derivanti dalla divisione per zero in fase di calcolo della covarianza.

4.2.1 Analisi Macro-Livello dei Database

Al fine di caratterizzare la robustezza e la resilienza del sistema Text-to-SQL sviluppato, la **Tabella C.7** riassume le metriche strutturali e i tassi di successo globali (cross-model) registrati sui 30 database facenti parte dei benchmark *Spider* e *BIRD*. L'indagine a macro-livello permette di isolare l'impatto combinato della complessità dello schema relazionale e della complessità intrinseca delle query sul *Success Rate* complessivo, evidenziando eventuali scostamenti tra la modalità statica basata su prompt testuale (**Success Rate text**) e la modalità dinamica guidata dalla connessione nativa al DBMS (**Success Rate db_conn**).

Dall'analisi sistematica del dataset emergono contesti applicativi in cui i modelli dimostrano di essere in grado di fornire risposte corrette, supportata da schemi relazionali compatti e richieste utente a bassa complessità sintattica.

Il database **poker_player** (*Spider*) rappresenta il punto di massimo successo per l'intero sistema, registrando un **Success Rate db_conn** del 94,58% e un **Success Rate text** del 72,92% (Tabella C.7). Tale comportamento è strutturalmente giustificata da fattori architetturali minimali: un numero di tabelle pari a $N_t = 2$, una densità relazionale controllata ($D_r = 5,5$) e, in particolare, l'indice di complessità delle query più basso registrato nel campione (Avg Query Complexity = 1,95). La distribuzione del corrispondente *Triplet Score* risulta infatti fortemente polarizzata verso compiti elementari (32 query *Low*, 8 *Medium*, 0 *High*).

Analogamente, i database **orchestra** (89,58% in **db_conn**), **singer** (90,56% in **db_conn**) ed **employee_hire_evaluation** (88,60% in **db_conn**) fanno intuire la tendenza per cui la combinazione di schemi ridotti ($N_t \leq 4$) e la sostanziale assenza di query ad alta complessità nel *Triplet Score* consente ai modelli di convergere efficacemente verso la struttura SQL corretta (Tabella C.7). In questi domini, lo sforzo di astrazione semantica richiesto è minimo, riducendo le probabilità di errore nell'allineamento tra le entità relazionali.

Una dettagliata e successiva attività di audit condotta sui log di sistema ha tuttavia fatto emergere una criticità sistematica, parzialmente esogena rispetto alla complessità strutturale finora discussa. Si è osservato che, persino in domini caratterizzati da un contenuto numero di entità relazionali e da richieste utente sintatticamente elementari, i modelli incorrono in fallimenti ripetuti a causa dell'istanza dei dati e, nello specifico, delle operazioni di filtraggio stringa nella clausola **WHERE**.

Quando la risoluzione del quesito in linguaggio naturale richiede l'applicazione di predicati operanti su attributi di tipo **VARCHAR** o **TEXT**, l'esatta stringa letterale (il valore del record) non viene quasi mai esplicitata nella consegna, né risulta deducibile dalla sola semantica dei metadati. In questo modo i Large Language Models si trovano costretti a formulare tentativi di computazione alla cieca. Tale dinamica innesca loop di raffinamento iterativo infruttuosi che, nella maggior parte delle istanze, determinano il definitivo fallimento della query per disallineamento rispetto alla ground truth, pur mantenendo una correttezza formale sotto il profilo sintattico.

Sul versante opposto, i decrementi prestazionali più severi si concentrano spesso all'interno del sotto-insieme appartenente al benchmark *BIRD*, storicamente noto per presentare sfide più vicine a scenari di produzione reali in termini di volume informativo e ambiguità del linguaggio naturale.

Il caso più critico è rappresentato dal database **thrombosis_prediction**, il quale crolla a un tasso di successo marginale del 2,25% in modalità **text**, recuperando parzialmente fino al 10,63% in modalità **db_conn** (Tabella C.7). Nonostante lo schema presenti un numero ridotto di tabelle ($N_t = 3$), esso mostra una densità relazionale critica, con una media di 21,33 colonne per tabella (64 attributi complessivi) e un elevato *Complexity Score* (3,87). La presenza di un numero massivo di attributi satura la finestra di contesto dei modelli in modalità **text**, innescando fenomeni di allucinazione o disallineamento semantico nella selezione dei campi.

Altrettanto complessi si rivelano i domini **california_schools** (**db_conn** al 23,78%) e **card_games** (**db_conn** al 27,14%) (Tabella C.7). In quest'ultimo scenario, l'overload informativo è guidato dalla scala assoluta dello schema relazionale, caratterizzato da 115 colonne totali distribuite su 6 tabelle. Pur mantenendo una complessità media delle query relativamente contenuta (2,25), la vastità del catalogo dati circostante disorienta l'euristica di join e di proiezione dei Large Language Models.

Un comportamento peculiare si osserva in **debit_card_specializing** e **financial**, i cui tassi in modalità **db_conn** si attestano rispettivamente al 19,79% e 15,88% (Tabella C.7). In questo caso, il principale fattore penalizzante non è ascrivibile allo schema, bensì all'elevata complessità intrinseca delle richieste: registrando un valore di **Avg Query Complexity** rispettivamente di 5,05 e 4,64, le distribuzioni dei loro *Triplet Score* mostrano una netta prevalenza di query classificate come *Medium* e *High*.

Il trend macroscopico più rilevante e matematicamente costante rintracciabile nei dati è l'ampio delta prestazionale a favore della modalità basata su connessione attiva al database (**db_conn**) rispetto a quella puramente testuale (**text**).

Come evidenziato dall'andamento complessivo dei test, il passaggio dalla modalità **text** alla modalità **db_conn** genera un incremento sistematico del *Success Rate* su tutte le istanze campionarie, con guadagni netti che superano frequentemente i 30-40 punti percentuali. Si vedano, a titolo esemplificativo, i seguenti casi emblematici (Tabella C.7):

- **course_teach**: evidenzia una transizione da un iniziale 43,89% (**text**) a un 78,33% (**db_conn**) [$\Delta = +34,44\%$].
- **cre_Doc_Template_Mgt**: si eleva dal 38,89% a un 78,17% [$\Delta = +39,28\%$].
- **museum_visit**: sperimenta un netto incremento dal 49,07% a un 87,96% [$\Delta = +38,89\%$].

Questo comportamento indica l'utilità della gestione dinamica del database da parte dell'orchestratore. Nella modalità **text**, l'immissione statica delle definizioni DDL nel prompt espone il modello a un rumore informativo costante e all'incapacità di risolvere i tipi di dato o i vincoli di integrità in tempo reale. Al contrario, l'architettura **db_conn** opera come un efficace vincolo euristico esterno: l'estrazione mirata dei metadati, la validazione dinamica del dialetto SQL e la facoltà di interrogare i cataloghi del DBMS forniscono al framework di generazione una consapevolezza contestuale. Questa sinergia mitiga le eccezioni a runtime e gli errori di sintassi, stabilizzando le performance anche a fronte di database caratterizzati da elevati indici di complessità.

4.2.2 Analisi Comparativa del Comportamento dei Modelli

Al fine di comprendere l'efficacia dell'introduzione di una connessione diretta al DBMS rispetto alla pipeline puramente testuale, è necessario analizzare nel dettaglio il comportamento di ciascun modello all'interno dei diversi domini operativi (Tabelle C.1.1–C.6.1). La disamina che segue mappa le reazioni delle architetture neurali strutturando il confronto lungo tre direttrici fondamentali: l'efficienza temporale, il numero di tentativi di raffinamento iterativo e l'anatomia dei fallimenti generativi (ripartiti in errori di sintassi, eccezioni a runtime e incongruenze semantiche).

L'introduzione della modalità **db_conn**, la quale prevede cicli iterativi di feedback basati sui messaggi d'errore nativi del DBMS, comporta una mutazione dei tempi complessivi di elaborazione.

I modelli **Codestral** e **Qwen3-Coder-Next** dimostrano la migliore ottimizzazione dei tempi nella modalità assistita (Tabelle C.4.1 e C.6.1). In particolare, **Qwen3-Coder-Next** fa registrare un tempo medio globale in modalità **db_conn** di soli 11,15 secondi (a fronte dei 5,35 secondi in modalità **text**), contenendo il decremento prestazionale temporale (Δ) a un sostenuto +5,80 secondi (Tabella C.6.1). **Codestral** segue a breve distanza con una latenza media in **db_conn** di 11,35 secondi ($\Delta = +6,09$ s) (Tabella C.4.1). Questi dati evidenziano la capacità di tali architetture open-weight di processare il contesto aggiuntivo e i feedback del database in modo rapido, minimizzando l'overhead computazionale del framework.

Al contrario, il picco di latenza computazionale viene registrato da **GPT-5-mini**, il quale fa registrare un tempo medio in **db_conn** di ben 25,05 secondi, raddoppiando i già elevati 11,58 secondi della

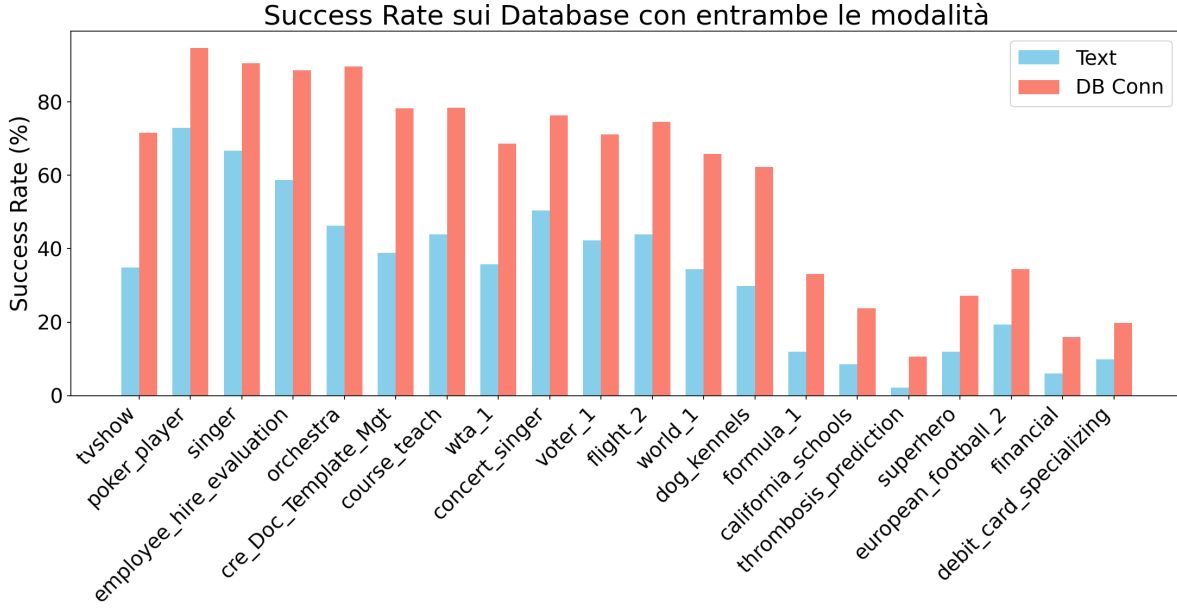


Figura 4.2: Numero di query risolte correttamente per database e per modalità.

modalità **text** ($\Delta = +13,47$ s) (Tabella C.2.1). Tale comportamento denota un approccio marcatamente più verboso o una maggiore complessità computazionale interna durante le fasi di raffinamento della query, traducendosi in un costo temporale che potrebbe risultare proibitivo in contesti di produzione in tempo reale. Anche **SQLCoder** mostra criticità evidenti, con una media in **db_conn** di 21,06 secondi (Tabella C.5.1); in questo scenario, tuttavia, il sovraccarico è giustificato dall'elevato numero di tentativi effettuati, a differenza del modello commerciale di OpenAI.

Nel database **california_schools**, **GPT-5-mini** evidenzia una controtendenza di rilievo: il tempo computazionale decresce da 32,76 secondi in modalità **text** a 21,16 secondi in **db_conn** ($\Delta = -11,60$ s) (Tabella C.2.1). Questo caso limite suggerisce che, in assenza di uno schema informativo reale, il modello rimanga vincolato in lunghi cicli di generazione di token esplorativi (allucinazioni di colonne o tabelle), mentre la precisione fornita dalla connessione diretta al DBMS ne accelera il processo di convergenza logica. Di contro, nel database **poker_player**, quasi tutte le architetture commerciali e avanzate (**GPT-4o**, **GPT-5-mini**, **Qwen3-Coder-Next**) convergono sulla query corretta entro un intervallo ridotto (tra i 5 e i 9 secondi totali in modalità **db_conn**) (Tabelle C.1.1, C.2.1, C.6.1), facendo intuire che la semplicità strutturale sia in grado di ridurre il costo del ciclo di feedback.

L'analisi del numero medio di tentativi necessari per giungere alla formulazione finale della query mette in luce la resilienza e la capacità di interpretazione dell'errore delle diverse intelligenze artificiali.

Qwen3-Coder-Next e **Codestral** appaiono come i modelli più diretti ed efficienti. **Qwen3-Coder-Next** richiede una media di appena 1,68 tentativi in modalità **db_conn** per concludere la sessione (con un Δ rispetto alla modalità **text** di soli +0,58 tentativi) (Tabella C.6.1). Questi dati indicano che, nella maggior parte dei casi, il modello genera la query corretta alla prima iterazione o necessita di un solo ciclo di correzione per sanare l'anomalia. **Codestral** si attesta su livelli analoghi, registrando 1,79 tentativi medi in modalità **db_conn** (Tabella C.4.1).

Sul versante opposto, **SQLCoder** mostra il comportamento più polarizzato, registrando una media globale di 4,31 tentativi in **db_conn** ($\Delta = +3,02$ tentativi rispetto a **text**) (Tabella C.5.1). Questo dato indica che **SQLCoder** satura quasi regolarmente il tetto massimo di tentativi consentiti dalla pipeline, incontrando notevoli difficoltà nel comprendere i messaggi d'errore restituiti dal DBMS. Il modello reitera variazioni della query senza riuscire a capitalizzare il feedback ricevuto, rimanendo intrappolato in loop generativi sterili.

Un comportamento peculiare si osserva nel database **thrombosis_prediction**, dove **GPT-5-mini** mostra un Δ negativo nei tentativi ($\Delta = -0,50$): se in modalità **text** effettuava in media 2,44 tentativi ciechi, in modalità **db_conn** si ferma a 1,93 (Tabella C.2.1). Tuttavia, l'incrocio con i dati sul

tasso di successo (stabile al 12,88%) indica anche che il modello interrompe prematuramente l'esplorazione logica o che i vincoli imposti dal feedback del DBMS lo portano a troncare la computazione senza esplorare alternative valide. Al contrario, nel database `real_estate_properties`, SQLCoder raggiunge un picco di 5,25 tentativi medi, traducendosi in un modesto 0% di *Success Rate* in modalità `db_conn` (Tabella C.5.1). Questo rappresenta il caso paradigmatico di un'architettura che esaurisce il budget computazionale senza mostrare alcuna sensibilità ai feedback di errore dello schema.

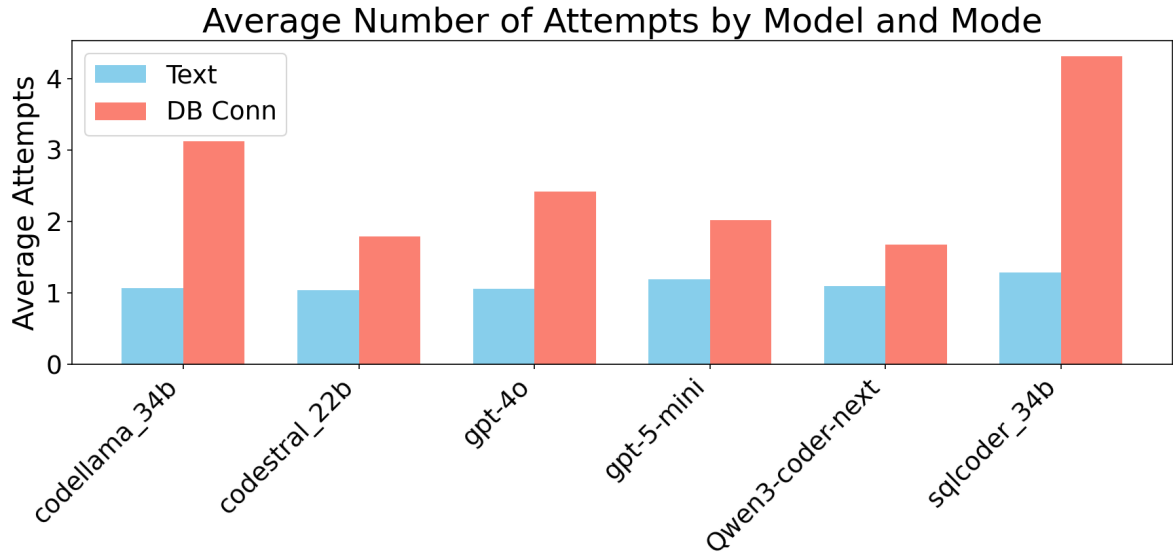


Figura 4.3: Numero medio di tentativi e tempo medio di esecuzione per modello.

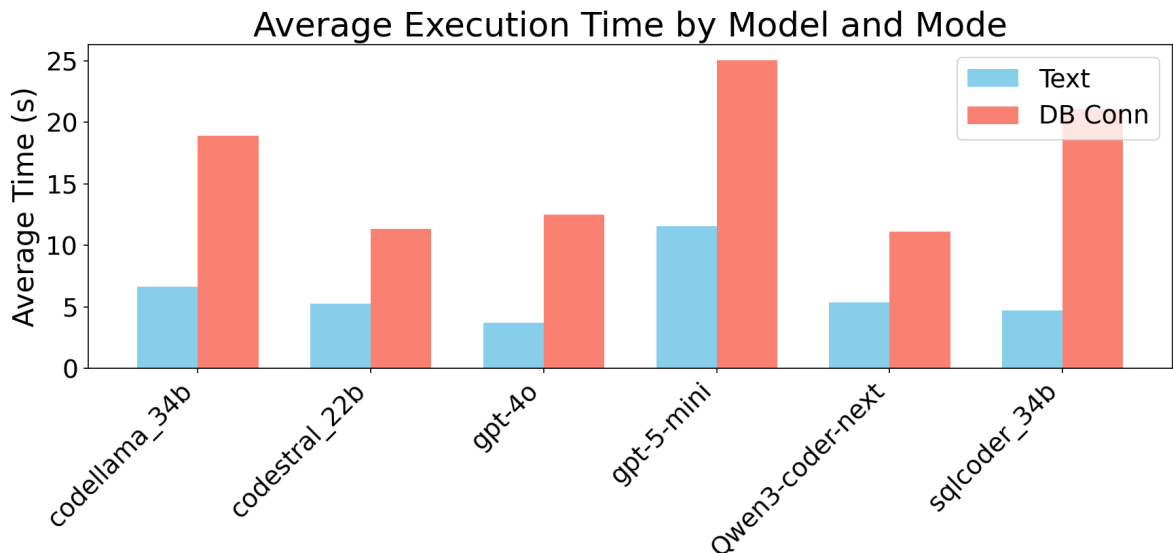


Figura 4.4: Numero medio di tentativi e tempo medio di esecuzione per modello.

La categorizzazione dei fallimenti (`Syntax`, `Runtime`, `Incorrect`) permette di mappare la natura profonda dei limiti di ciascun modello (Tabelle C.1.1–C.6.1). Un'evidenza macroscopica emerge immediatamente dall'analisi dei dati: per tutti e sei i modelli esaminati, gli errori di sintassi pura (`Syntax`) in modalità `db_conn` sono pari a zero. La pipeline di ingegneria del prompt e la natura intrinseca dei modelli attuali sono riuscite a rimuovere le anomalie di parsing elementari in tutte le sessioni di test; la sfida valutativa si sposta interamente sulle eccezioni a runtime e sugli errori semantici.

Gli errori di runtime, ovvero query sintatticamente valide che falliscono durante l'esecuzione sul database a causa di identificatori errati, tabelle inesistenti o join ambigue, variano sensibilmente tra i modelli. **GPT-5-mini** e **Qwen3-Coder-Next** guidano la classifica della sicurezza esecutiva. **GPT-5-mini** commette solo 38 errori di runtime globali in modalità **db_conn** (a fronte dei 2021 registrati in modalità **text**), mettendo in luce un decremento significativo ($\Delta = -1983$ errori) (Tabella C.2.1). **Qwen3-Coder-Next** si attesta a soli 55 errori di runtime in **db_conn** ($\Delta = -291$) (Tabella C.6.1). Questo rivela l'efficacia euristica del modulo di connessione, che permette a tali modelli di correggere istantaneamente le sviste di nomenclatura strutturale dello schema relazionale. Di contro, **SQLCoder** fallisce ripetutamente il debugging a runtime, totalizzando 1016 errori in **db_conn** (Tabella C.5.1). Sebbene riduca i fallimenti rispetto alla modalità **text** (1712 errori, $\Delta = -696$), la cifra assoluta rimane critica, testimoniando l'incapacità del modello di elaborare la natura delle eccezioni SQL (ad esempio, l'errore *Unknown column* in una clausola **WHERE**).

Gli errori classificati come **Incorrect** rappresentano il fallimento più insidioso nell'interazione Text-to-SQL: la query viene eseguita dal DBMS senza sollevare eccezioni, ma restituisce un set di dati errato rispetto ai requisiti logici espressi dall'utente. L'introduzione della metrica di *Incorrect Rate* consente di isolare la pura capacità di comprensione semantica del modello, separandola dalla sua abilità nel generare codice formalmente valido.

Il modello **GPT-4o** possiede un comportamento antitetico tra le due modalità (Tabella C.1.1). In configurazione **text**, il modello registra un numero assoluto di errori semantici relativamente basso, dinamica dovuta esclusivamente all'elevato tasso di fallimenti a runtime (2191 query bloccate), che riduce in maniera considerevole il campione di query giunte a esecuzione effettiva. Infatti, laddove la query riesce a essere eseguita, l'*Incorrect Rate* locale tocca picchi critici (pari al 65% in **california_schools** e al 61,54% in **superhero**). Nel passaggio alla modalità **db_conn**, l'introduzione del contesto strutturato azzerava quasi i runtime (crollati a 102), ma fa traslare gli errori semantici assoluti a 995 ($\Delta = +910$) (Tabella C.1.1).

Al contrario, **Qwen3-Coder-Next** e **Codestral** suggeriscono una discreta robustezza logica, mantenendo l'*Incorrect Rate* bilanciato tra le due modalità e, in diversi scenari, registrando persino un incremento della percentuale di query corrette in **db_conn** (ad esempio, **Qwen3-Coder-Next** riduce l'incidenza di errore in **superhero**) (Tabelle C.4.1 e C.6.1). **Qwen3-Coder-Next** si attesta a 976 errori semantici in **db_conn** ($\Delta = +196$ rispetto a **text**) (Tabella C.6.1), mentre **Codestral** ne totalizza 1040 ($\Delta = +124$) (Tabella C.4.1). Tali dati attestano come questi modelli specialistici non si limitino a sfruttare lo schema per risolvere la sintassi, ma ne assimilino i vincoli relazionali per raffinare la precisione dell'output logico.

Nei database caratterizzati da un'elevata densità informativa e da un lessico settoriale, si assiste a una massiccia conversione degli errori a runtime in errori semantici, specialmente nelle architetture di ultima generazione. Emblematico è il caso di **GPT-5-mini** sul database **toxicology**: gli errori di runtime si azzerano completamente in modalità **db_conn** (rispetto ai 144 in **text**), ma l'*Incorrect Rate* si attesta stabilmente sul 100% delle query eseguite (Tabella C.2.1), evidenziando una traslazione totale del fallimento dalla forma al contenuto logico.

A seguito di un'ispezione approfondita condotta sui log di sistema, è stato possibile isolare la causa determinante di tale fenomeno, riscontrabile anche in domini apparentemente meno complessi o con un contenuto numero di tabelle e colonne (quali **real_estate_properties**, **card_games**, **thrombosis_prediction** e **superhero**). In questi scenari, la risoluzione del task richiede l'applicazione di filtri stringenti all'interno della clausola **WHERE** su attributi di tipo **VARCHAR**. Tuttavia, i valori categoriali o testuali esatti necessari a soddisfare il predicato algebrico non vengono specificati nella richiesta in linguaggio naturale, né sono desumibili dalla mera struttura dello schema relazionale. Di conseguenza, i Large Language Models eseguono ripetuti tentativi alla cieca inserendo stringhe letterali plausibili ma fattualmente errate. Il DBMS esegue con successo la query (azzerando i runtime error) ma restituisce un set di dati vuoto o non conforme a quelli della *gold query*, sancendo il fallimento semantico della generazione e saturando inutilmente il budget di raffinamento iterativo.

In conclusione, l'analisi di dettaglio rivela che **Qwen3-Coder-Next** e **Codestral** rappresentano le soluzioni più bilanciate ed efficienti, in quanto capaci di convergere rapidamente con ridotti tempi di elaborazione, richiedere un numero esiguo di iterazioni e mostrare un controllo ottimale sui runtime

senza incrementare in modo anomalo la quota di errori logici. Parallelamente, i modelli proprietari GPT-4o e GPT-5-mini beneficiano in modo non indifferente della modalità connessa per abbattere i propri errori di runtime, i quali nella modalità `text` ne azzeravano quasi completamente l'usabilità; tuttavia, queste architetture pagano il prezzo di un sensibile aumento dei tempi di elaborazione e di una forte traslazione dei fallimenti verso la dimensione semantica (`Incorrect`).

Infine, modelli come `SQLCoder` e `CodeLlama`, con quest'ultimo caratterizzato da un *Success Rate* generale ancorato al 34,81% in modalità `db_conn` (Tabella C.3.1), appaiono inadatti a contesti di interazione dinamica con il DBMS, poiché faticano a interpretare correttamente i messaggi di errore restituiti dal sistema e finiscono per saturare tutti i tentativi disponibili senza mostrare una reale capacità di auto-correzione.

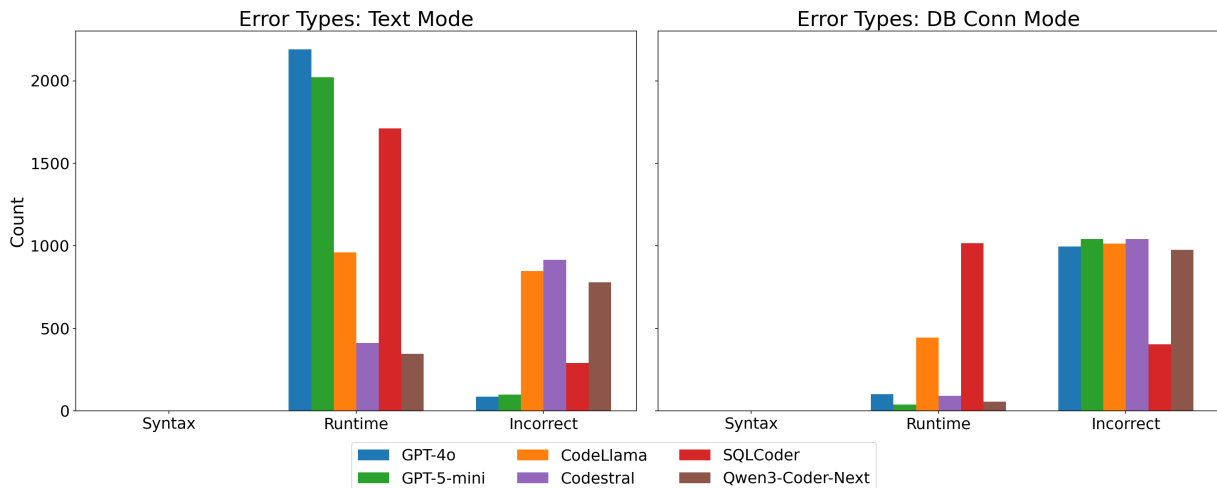


Figura 4.5: Confronto delle tipologie di errore riscontrate nei diversi modelli.

4.2.3 Analisi Inferenziale e Studio delle Correlazioni Statistiche

L'analisi dei coefficienti di correlazione lineare di Pearson (ρ), deputato a misurare l'intensità delle relazioni lineari, e di correlazione per ranghi di Spearman (r_s), orientato a intercettare relazioni monotoniche anche non lineari, consente di caratterizzare l'impatto reale di due variabili esogene sulla probabilità di successo del codice SQL generato (codificato come variabile binaria $Y \in \{0, 1\}$, dove 1 indica il successo e 0 il fallimento). Le variabili d'indagine considerate sono il numero di tentativi di raffinamento (*attempts*) impiegati dall'orchestrazione e l'indice di complessità normalizzato del database (*complexity score*).

I dati estratti relativi alla correlazione tra il numero di iterazioni e l'esito della generazione mostrano trend consistenti tra le sei architetture esaminate (`CodeLlama`, `Codestral`, `GPT-4o`, `GPT-5-mini`, `Qwen3-Coder-Next`, `SQLCoder`) (Tabelle C.1.2–C.6.2), svelando dinamiche cruciali sulla capacità delle singole strutture di capitalizzare il feedback di esecuzione del database (`db_conn`) rispetto al contesto puramente testuale (`text`).

Sotto il profilo metodologico, data la codifica binaria della variabile target, una correlazione negativa indica che l'aumento dei tentativi di raffinamento è associato a una contrazione sistematica della probabilità di successo. Questo segno, ampiamente diffuso nel dataset, indica come i modelli tendano a concentrare l'efficacia risolutiva nelle primissime iterazioni. Al contrario, il prolungarsi della sessione verso i limiti superiori di raffinamento non produce benefici incrementali, mostrando che l'algoritmo tende a rimanere vincolato in loop generativi sterili o a divergere progressivamente dall'intento semantico originario.

I modelli `GPT-4o` e `CodeLlama` (quest'ultimo in contesti specifici quali il database `battle_death`) rivelano coefficienti di correlazione negativi robusti e statisticamente significativi ($p\text{-value} < 0,05$) (Tabelle C.1.2 e C.3.2). In particolare, per `GPT-4o`, la convergenza verso una correlazione negativa marcata nella configurazione `db_conn` certifica che, sebbene il sistema sia predisposto per l'auto-correzione, l'esaurimento dei tentativi si traduce in un decremento della probabilità di successo; se l'errore non viene sanato immediatamente nel primo ciclo di feedback, i messaggi del DBMS nelle

iterazioni successive faticano a riallineare la query, agendo talvolta come rumore informativo che induce il modello a tentare variazioni strutturali non conformi.

Al contrario, **Codestral** e **Qwen3-Coder-Next** evidenziano dinamiche parzialmente differenti in scenari localizzati. Nel database **battle_death**, ad esempio, presentano una correlazione debolmente positiva, sebbene non statisticamente rilevante ($p\text{-value} > 0,05$), con un coefficiente di Pearson pari a $+0,1155$ per **Codestral** (Tabella C.4.2) e a $+0,1615$ per **Qwen3-Coder-Next** (Tabella C.6.2). Tale lieve inversione di tendenza indica una sporadica capacità di queste architetture di beneficiare dei tentativi avanzati per convertire un fallimento iniziale in successo, sebbene il dato non trovi una spinta statistica sufficiente a scardinare il trend negativo globale.

In questo scenario, il delta computazionale individua la misura in cui la connessione dinamica al database modifichi l'efficacia dei tentativi rispetto al mero ragionamento statico sulle istruzioni DDL. In modelli verticali come **SQLCoder**, si osserva un comportamento asimmetrico: nel dominio **battle_death** il modello registra una correlazione negativa significativa in modalità **db_conn** ($\rho = -0,5375$) che si azzerà nel passaggio alla modalità **text** a causa della mancanza di dati validi (valori contrassegnati come *Non Rilevabili* o assenti) (Tabella C.5.2). In tale scostamento si riscontra la dipendenza di **SQLCoder** dal feedback d'errore interattivo per tentare una qualsiasi forma di movimento nella pipeline, pur confermando che l'accumulo di iterazioni deteriora linearmente l'accuratezza finale.

Il modello **GPT-5-mini** mostra un andamento emblematico su database ridotti o specifici (Tabella C.2.2), dove l'assenza di correlazione misurabile o valori prossimi allo zero indicano che l'architettura soffre di un'alta varianza su schemi compatti, in cui un singolo fallimento iniziale tende a compromettere irrevocabilmente l'evoluzione statistica dei tentativi successivi senza mostrare un pattern di declino o crescita lineare definito.

Esaminando i dati aggregati sulla correlazione tra la complessità (sia della query che dello schema) e il tasso di successo (Tabelle C.1.3–C.6.3), emergono macro-tendenze fondamentali che accomunano l'intero corpus di test.

La prima evidenza è rappresentata dalla correlazione negativa generalizzata: la quasi totalità dei coefficienti aggregati di Pearson e Spearman presenta valori negativi significativi, supportati da un $p\text{-value}$ matematicamente prossimo allo zero (con ordini di grandezza compresi tra 10^{-15} e 10^{-49}). I $p\text{-value}$ molto bassi indicano una forte evidenza statistica di associazione nel campione analizzato, pur con le cautele dovute alla dipendenza tra osservazioni appartenenti allo stesso database e allo stesso modello. Tale dato mette in luce il fatto che all'aumentare della complessità della richiesta o della densità dello schema relazionale, la probabilità di successo dei modelli diminuisce costantemente.

Dall'analisi dei valori medi emerge inoltre il vantaggio strutturale della modalità **db_conn** rispetto a quella **text**. La complessità dello schema mostra una correlazione negativa più marcata nella modalità connessa; **CodeLlama**, a titolo esemplificativo, registra un coefficiente di $-0,2928$ contro il $-0,2267$ della modalità testuale (Tabella C.3.3). Tale dinamica indica che la complessità dello schema penalizza maggiormente i modelli quando questi sono vincolati a una connessione database reale, un fenomeno riconducibile alla rigidità sintattica richiesta per l'esecuzione diretta sul DBMS rispetto alla maggiore tolleranza tipica del parsing testuale astratto.

Si rileva, infine, un notevole allineamento tra i coefficienti di Pearson e Spearman, i quali seguono traiettorie quasi identiche. Questa convergenza attesta la robustezza statistica del dato, indicando che il declino delle performance non è causato da anomalie isolate o outliers, ma rappresenta un trend monotonicamente e strutturalmente intrinseco al comportamento dei modelli.

I modelli esaminati non reagiscono in modo uniforme ai fattori di complessità, ma possono essere ripartiti in tre distinte classi di resilienza:

1. **Architetture Specialistiche di Vecchia Generazione (CodeLlama, SQLCoder):** **CodeLlama** mostra la sensibilità più severa e lineare alla complessità dello schema, registrando un coefficiente di Pearson in modalità **db_conn** pari a $-0,2928$ nella media delle colonne (Tabella C.3.3). Il modello manifesta criticità evidenti nell'orchestrazione di schemi estesi e mantiene un trend rigidamente negativo prossimo a $-0,17$ in entrambe le modalità operative. **SQLCoder** esibisce un comportamento affine, sebbene con una penalizzazione leggermente inferiore sulla complessità dello schema ($\rho = -0,1798$ in **db_conn**) (Tabella C.5.3). In questo caso, il delta tra le modalità

`db_conn` e `text` si attesta a un marginale $-0,0058$, evidenziando come il modello risenta della complessità architetturale a prescindere dall'interfaccia di interazione adottata.

2. **Classe Intermedia ad Alte Prestazioni (Codestral, Qwen3-Coder-Next):** Codestral registra una delle correlazioni negative più marcate sulla complessità complessiva della query, con un valore di Spearman in modalità `db_conn` pari a $-0,2135$ (Tabella C.4.3). Questo dato denota che, nonostante si tratti di un'architettura nativa per il codice, le richieste strutturalmente pesanti ne minano profondamente l'accuratezza. Al contrario, Qwen3-Coder-Next dimostra un comportamento peculiare: l'impatto negativo della complessità dello schema in modalità `text` ($-0,2186$) è superiore rispetto alla modalità `db_conn` ($-0,1907$), generando un delta positivo di $+0,0279$ (Tabella C.6.3). Tale inversione suggerisce una spiccata capacità del modello di beneficiare della connessione al database attivo per validare e gestire gli schemi complessi.
3. **Modelli di Frontiera e Ragionamento (GPT-4o, GPT-5-mini):** GPT-4o si dimostra l'architettura più resiliente alla complessità in configurazione testuale; nella valutazione sul totale del modello in modalità `text`, registra un coefficiente di Pearson pressoché nullo ($+0,0153$) associato a un p -value non significativo di $0,4530$ (Tabella C.1.3). Di conseguenza, GPT-4o è l'unico modello le cui performance su testo non sono linearmente influenzate dalla complessità della query, sebbene il passaggio alla modalità `db_conn` riporti la correlazione a un valore negativamente significativo ($-0,1909$), evidenziando l'impatto dei vincoli reali del DBMS. GPT-5-mini segue l'andamento del predecessore mostrando una forte contraction dell'impatto della complessità in modalità `text` ($\rho = -0,0582$), ma palesa comunque una fragilità marcata quando si scontra con la complessità strutturale dello schema in modalità `db_conn`, dove il valore di Pearson per la media delle colonne scende a $-0,1682$ (Tabella C.2.3).

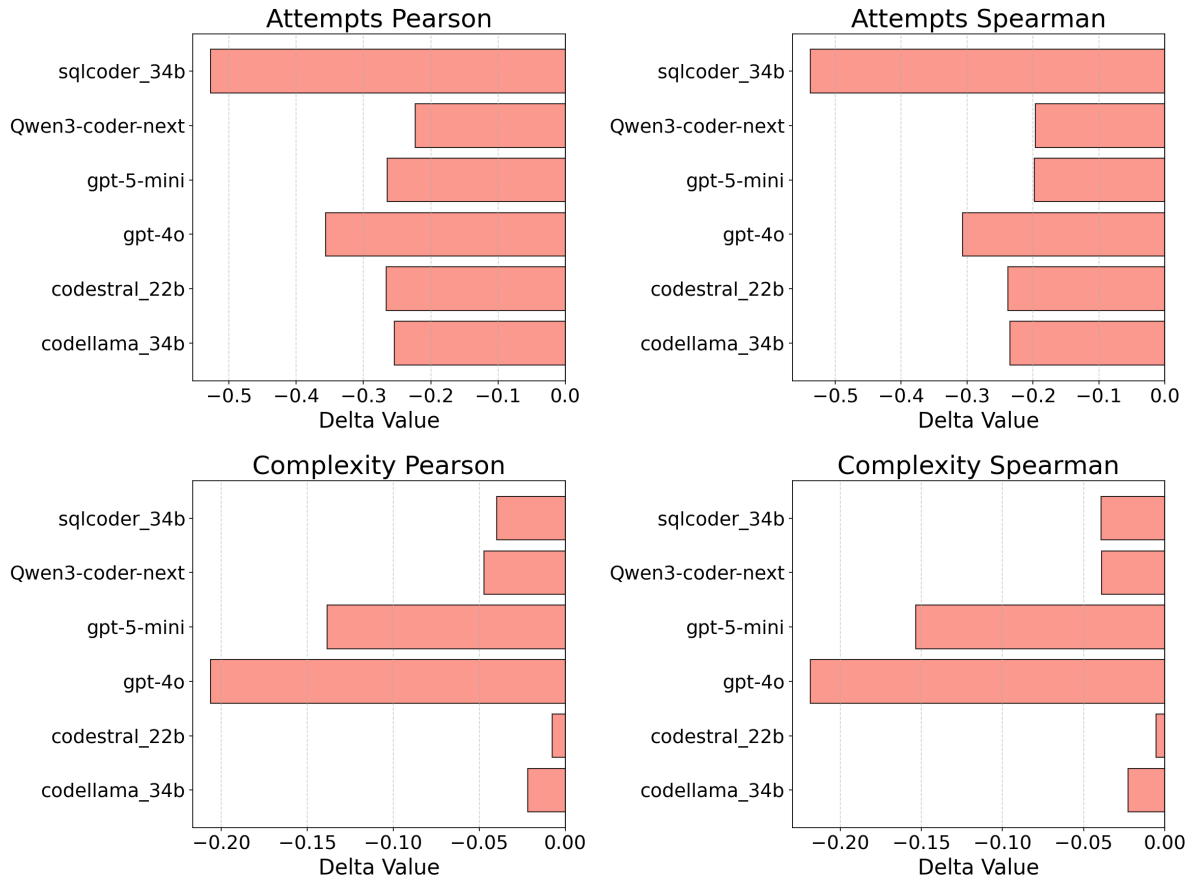


Figura 4.6: Variazione delle correlazioni tra complessità del database e metriche sperimentali.

La disamina dei singoli database evidenzia dinamiche locali che deviano dal trend globale, offrendo significativi spunti analitici.

Una prima evidenza è rappresentata dai domini definiti “*Killer Database*”, quali `dog_kennels`, `world_1`, `toxicology` e `codebase_community`. In questi contesti, la totalità dei modelli mostra una correlazione negativa solida e statisticamente convalidata. Nel database `dog_kennels`, ad esempio, i coefficienti oscillano costantemente tra $-0,25$ e $-0,41$, a dimostrazione del fatto che tali schemi e le relative query costituiscono il principale collo di bottiglia per l’attuale tecnologia Text-to-SQL.

Al contrario, in contesti specifici si assiste al fenomeno controintuitivo delle correlazioni positive anomale, come nei casi di `poker_player`, `employee_hire_evaluation` e `cre_Doc_Template_Mgt`. All’interno di `poker_player`, CodeLlama registra un coefficiente di Pearson in modalità `db_conn` pari a $+0,3549$ e SQLCoder raggiunge $+0,4155$, mentre `employee_hire_evaluation` mostra in CodeLlama un valore di $+0,3753$ in modalità `text`. Sotto il profilo tecnico, una correlazione positiva implica che all’aumentare della complessità i modelli abbiano ottenuto un tasso di successo superiore. Tale paradosso si ipotizza possa essere causato dalla struttura dei dati di test: nei database menzionati, le query catalogate come ad alta complessità presentano pattern sintattici ricorrenti, funzioni di aggregazione standardizzate o clausole di JOIN obbligate su chiavi esplicite su cui i modelli potrebbero avere eseguito un overfitting in fase di addestramento. Viceversa, le richieste classificate come semplici esibiscono frequenti ambiguità linguistiche nel testo naturale che inducono i modelli in errore.

Un ulteriore caso limite è rappresentato da `voter_1` in GPT-4o, dove si registra una elevata polarizzazione tra le due modalità operative: il modello mostra una correlazione negativa in `db_conn` ($-0,3434$) a fronte di una correlazione fortemente positiva in modalità `text` ($+0,7447$ con un p -value di $0,0014$). Tale comportamento indica che il modello genera query complesse che testualmente risultano coerenti e rigorose, giustificando l’alta correlazione su testo, ma che all’atto pratico falliscono l’esecuzione sul database reale a causa di disallineamenti di nomenclatura o vincoli di schema non intercettati.

Infine, per i modelli commerciali (GPT-4o e GPT-5-mini), si rileva la presenza di coefficienti contrassegnati come *Non Rilevabili* (ND), circostanza ricorrente su database quali `museum_visit`, `network_1` e `orchestra`. Questa condizione si verifica quando il tasso di successo del modello nel database specifico è pari al 100% o allo 0% su tutti i livelli di complessità. Venendo a mancare la varianza nei risultati a causa della costanza degli esiti, l’algoritmo computazionale azzerava il denominatore nella formula della covarianza, producendo una divisione per zero che rende formalmente impossibile determinare la correlazione statistica.

4.2.4 Conclusioni dell’Analisi

In conclusione, l’impianto sperimentale strutturato attorno al modulo `summarize_models.py` ha permesso di mappare i confini operativi del sistema con tecnologia Text-to-SQL di fronte alla complessità dei dati. L’analisi inferenziale condotta tramite i coefficienti di Pearson e Spearman ha consentito di individuare la presenza di alcuni trend negativi strutturali e generalizzati: la densità degli schemi relazionali e la complessità intrinseca delle query rappresentano i principali fattori esogeni di degradazione delle performance predittive.

Tuttavia, i dati di test evidenziano come l’introduzione di un’architettura basata su connessione interattiva al database (`db_conn`) costituisca una strategia di mitigazione determinante. Sebbene tale modalità imponga un inevitabile overhead in termini di latenza temporale e richieda cicli di feedback iterativi, essa azzerava di fatto gli errori di sintassi pura e abbatté in modo significativo le eccezioni a runtime, offrendo ai modelli una consapevolezza contestuale superiore, altrimenti preclusa nella configurazione statica su prompt testuale (`text`).

La disamina empirica delinea una sorta di gerarchia tra i modelli: mentre le architetture open-weight specialistiche come Qwen3-Coder-Next e Codestral mostrano un bilanciamento tra tempi di computazione, accuratezza logica e interpretazione del feedback di errore, i modelli proprietari di frontiera (GPT-4o e GPT-5-mini) massimizzano l’abbattimento dei runtime a prezzo di una marcata traslazione dei fallimenti verso la dimensione semantica (*Incorrect*). Al contrario, le soluzioni di precedente generazione palesano una sostanziale inidoneità ai contesti dinamici, saturando i tentativi disponibili senza mostrare una reale capacità di auto-correzione. I risultati ottenuti non solo validano l’efficacia metodologica del framework sviluppato, ma forniscono coordinate empiriche fondamentali

per il futuro design di pipeline RAG e agenti autonomi applicati a sistemi di gestione di database relazionali complessi.

5 Conclusioni

5.1 Raggiungimento degli obiettivi

Il presente lavoro di tesi si è posto l’obiettivo di superare la barriera tecnica tra il linguaggio naturale umano e la rigidità sintattica delle basi di dati relazionali. L’attività di ricerca e sviluppo si è concretizzata nella realizzazione di un framework basato su Python, strutturato per agire come un intermediario intelligente capace di orchestrare l’intero ciclo di vita di una richiesta text-to-SQL.

L’obiettivo principale è stato perseguito attraverso l’implementazione di una serie di moduli interconnessi, la cui efficacia è stata verificata in fase sperimentale.

Di seguito vengono analizzate nel dettaglio le soluzioni tecniche ideate per il raggiungimento degli obiettivi.

L’obiettivo di estrarre e formalizzare la conoscenza del database è stato raggiunto attraverso lo sviluppo della classe **SchemaOrchestrator**. Questo componente gestisce l’acquisizione del metadato tecnico operando secondo due direttrici: l’estrazione diretta tramite connessione al database (**db_conn**) o l’elaborazione di descrizioni testuali fornite dall’utente tramite Large Language Models. Il risultato di tale processo è la creazione di un oggetto **Schema**, che funge da rappresentazione strutturata e centralizzata del patrimonio informativo del database, fornendo una base solida per le successive fasi di generazione.

Per ovviare ai limiti della finestra di contesto dei modelli linguistici, è stato implementato un sistema di gestione del contesto basato su architettura RAG. Il componente **SchemaStore** agisce come un database vettoriale in cui lo schema viene frammentato e immagazzinato sotto forma di documenti. Grazie all’integrazione di un retriever, il sistema è in grado di analizzare la richiesta in linguaggio naturale dell’utente e isolare dinamicamente solo le porzioni di schema (tabelle e relazioni) realmente pertinenti al contesto della query. Questo approccio non solo ottimizza il consumo di token, ma è in grado anche di ridurre il “rumore” informativo, migliorando l’accuratezza del modello.

La flessibilità del sistema nella generazione del codice SQL è fornita dal **PromptBuilder**. Questo modulo è responsabile della costruzione di prompt dinamici che si adattano a molteplici variabili: la tipologia di sorgente dati e le specifiche necessità sintattiche del dialetto SQL di destinazione. Il **PromptBuilder** integra in un unico template la richiesta dell’utente, il contesto estratto dallo **SchemaStore** e, in fase di raffinamento, i feedback derivanti dai fallimenti precedenti, consentendo una comunicazione coerente e contestualizzata con il modello linguistico.

Il requisito di robustezza della query prodotta è stato soddisfatto mediante un ciclo iterativo di validazione e raffinamento gestito dal **QueryOrchestrator**. Ogni query generata viene sottoposta a una duplice verifica: una validazione sintattica immediata (tramite librerie di parsing come **sqlglot**) e, ove possibile, una validazione semantica attraverso l’esecuzione reale sul database. In caso di errore (sia esso di sintassi o di runtime), interviene la logica di **LLMFeedback**, che analizza l’eccezione e produce un’indicazione correttiva che viene reiniettata nel prompt per il tentativo successivo. Questo meccanismo di “auto-correzione” permette al sistema di imparare dai propri errori all’interno della medesima sessione, incrementando significativamente il tasso di successo finale delle interrogazioni generate.

L’ultimo requisito essenziale, finalizzato a rendere il framework fruibile, modulare e facilmente integrabile all’interno di ecosistemi applicativi di terze parti, è stato conseguito attraverso la progettazione e l’implementazione di uno strato di servizi web basato sul framework **FastAPI**. Sfruttando le caratteristiche native di asincronia, validazione dei tipi e generazione automatica della documentazione offerte da questa tecnologia, l’intera logica di business è stata esposta tramite un’architettura ad **API RESTful** solida e ad alte prestazioni.

La modularità del sistema si riflette a livello infrastrutturale nell’adozione di una strategia di routing distribuita. Il punto di ingresso dell’applicazione (**main.py**) garantisce la massima interoperabilità frontend e orchestra l’inclusione di router dedicati ed isolati, nello specifico **schema** e **query**.

Questo approccio assicura una netta separazione delle responsabilità e una manutenibilità ottimale del codice sorgente. Il `router schema.py` centralizza gli endpoint dedicati alla gestione del ciclo di vita dei metadati. Esso espone le rotte per l'estrazione diretta delle strutture relazionali da istanze MySQL remote (`/schema/mysql`), per la generazione sintetica di schemi a partire da testo non strutturato mediante modelli linguistici (`/schema/text`) e per il raffinamento iterativo delle definizioni esistenti (`/schema`). Il `router query.py` si occupa invece di governare i flussi operativi di traduzione del linguaggio naturale. Al fine di massimizzare la flessibilità del framework in contesti di test o di produzione offline, sono stati ingegnerizzati due flussi logici distinti: l'endpoint `/queries/text`, deputato alla sola generazione pura del codice SQL isolando la componente LLM da interazioni esterne, e l'endpoint `/queries/mysql`, il quale estende il precedente paradigma abilitando l'interconnessione con il DBMS, l'esecuzione della query transazionale e la serializzazione formale dei record risultanti.

In sintesi, la traduzione dei requisiti iniziali in un'architettura software funzionante ha dimostrato che l'integrazione tra modelli di linguaggio avanzati e sistemi di recupero delle informazioni rappresenta una soluzione robusta per democratizzare l'accesso ai dati, riuscendo ad ottenere al contempo sia la sicurezza che la correttezza formale delle interrogazioni eseguite.

5.2 Vantaggi e svantaggi

La valutazione sistematica delle prestazioni ha permesso di delineare con precisione i benefici introdotti dall'architettura proposta, nonché i vincoli tecnologici che ancora ne limitano l'efficacia in scenari di produzione complessi.

Il principale valore aggiunto del sistema risiede nella sua capacità di auto-correzione iterativa, poiché l'integrazione della modalità `db_conn` trasforma il framework da un semplice traduttore statico a un agente dinamico in grado di avvicinarsi sempre di più all'obiettivo finale.

Questa architettura favorisce una netta riduzione delle allucinazioni sintattiche, come dimostrato da una padronanza grammaticale del linguaggio SQL pressoché assoluta; l'assenza di errori di sintassi suggerisce infatti la validità dei template di prompt e dei vincoli strutturali imposti ai modelli portando la media generale dei successi da 23,78% a 45,33%. Al contempo, si rileva un'elevata efficacia del feedback loop: per modelli specializzati come SQLCoder il sistema agisce come un catalizzatore di precisione, poiché anche laddove la prima generazione fallisce, il meccanismo di cattura degli errori di runtime e la loro successiva re-iniezione nel prompt permettono di "raddrizzare" la query, garantendo il successo finale (da 16,39% a 25,39%).

L'analisi ha inoltre evidenziato la versatilità dei modelli open-source, mostrando come soluzioni quali Codestral e Qwen3-Coder-Next siano in grado di competere con architetture proprietarie del calibro di GPT-4o in contesti zero-shot, offrendo un'alternativa valida, economica e orientata alla privacy per l'interrogazione di schemi non eccessivamente densi. Per i primi due si osserva 48,46% e 55,77% rispettivamente mentre per i due successivi, 54% e 61,54%. Di conseguenza, il sistema raggiunge l'obiettivo della democratizzazione del dato, abbattendo la barriera tecnica per database con complessità moderata e permettendo l'estrazione di informazioni complesse, quali join e aggregazioni, senza la necessità di conoscere lo specifico dialetto SQL.

Nonostante i successi prestazionali, emergono criticità strutturali legate principalmente alla gestione del contesto e alla latenza operativa.

Il limite strutturale primario è rappresentato dalla sensibilità alla densità dello schema, specialmente nei database ampio come *california_schools* con 89 colonne totali, dove un numero elevato di attributi per singola tabella tende a saturare il contesto del modello. In questi scenari, il sistema fatica a distinguere tra termini semanticamente simili, producendo di conseguenza errori di runtime legati a nomi di colonne errati.

A questa barriera architetturale si aggiunge un'insidiosa vulnerabilità legata al fenomeno del Value Mapping, emersa dall'ispezione approfondita dei log di sistema su domini caratterizzati da un limitato punteggio sulla complessità generale ma da un severo tasso di fallimento logico (quali *real_estate_properties*, *card_games*, *toxicology*, *thrombosis_prediction* e *superhero*). In questi contesti, la corretta esecuzione della query richiede l'applicazione di filtri restrittivi nella clausola `WHERE` su attributi di tipo `VARCHAR`. Tuttavia, i valori categoriali o testuali esatti necessari a soddisfare il predicato algebrico non sono specificati nella richiesta in linguaggio naturale, né sono estraibili dai metadati strutturali. Privi

di un meccanismo di indicizzazione dei contenuti del database, i modelli eseguono ripetuti tentativi alla cieca inserendo costanti letterali plausibili ma fattualmente errate; in questo modo il DBMS esegue la query con successo azzerando i runtime error, ma restituisce un set di dati vuoto. Tale dinamica provoca una massiccia traslazione dei fallimenti verso la dimensione semantica, esaurendo inutilmente il budget di raffinamento iterativo.

A questo si sommano i limiti su latenza e costi computazionali, poiché l'adozione di un approccio multi-tentativo, sebbene efficace per incrementare la precisione, penalizza in modo significativo la velocità di risposta; modelli come GPT-5-mini, pur risultando accurati, presentano tempi di inferenza che possono superare i 50 secondi su database complessi, rendendo l'esperienza utente meno fluida in contesti operativi in tempo reale.

Si osserva inoltre una fragilità semantica in assenza di connessione nella modalità *text*, in cui il sistema dipende totalmente dalla qualità della documentazione dello schema. Senza l'ausilio della connessione diretta al database, la capacità di ricostruire relazioni logiche profonde cala, come dimostrato dal caso limite del database *orchestra*, evidenziando una dipendenza critica dall'introspezione dinamica, possedendo un delta tra le due modalità di 43,33%. Infine, l'analisi ha rilevato un'efficacia limitata del processo di raffinamento sui modelli generalisti; alcune architetture, pur essendo avanzate, non traggono benefici significativi dai tentativi aggiuntivi, mostrando una correlazione tra tentativi e successo vicina allo zero (esempio: Qwen3-Coder-Next). Questo fenomeno suggerisce che per tali agenti l'errore iniziale sia spesso di natura logica e, pertanto, non facilmente correggibile attraverso il solo feedback testuale fornito dal database.

5.3 Prospettive future

Sulla base delle criticità emerse in fase di valutazione sperimentale, sono state identificate diverse direttrici di ricerca e ottimizzazione ingegneristica volte a estendere le capacità e la robustezza del framework proposto.

Per ovviare al crollo delle prestazioni riscontrato nei database ad alta densità (caratterizzati da un elevato numero di attributi per singola tabella), una naturale evoluzione del sistema prevede l'introduzione di un modulo di Reranking integrato nello **SchemaStore**, ad esempio mediante l'impiego di modelli Cross-Encoder. Questo approccio consentirebbe un filtraggio del contesto a due stadi: un primo recupero preliminare basato sulla similarità coseno degli embedding vettoriali, seguito da una fase di riordinamento semantico fine. Tale strategia garantirebbe l'iniezione nel prompt del solo sottoinsieme di colonne strettamente necessarie alla risoluzione del quesito, riducendo il consumo di token nella finestra di contesto e azzerando quasi totalmente gli errori di Schema Linking.

Le criticità emerse nell'ambito del value mapping hanno evidenziato come, in assenza di informazioni puntuali sul contenuto dei record, i modelli linguistici non dispongano di elementi sufficienti per formulare correttamente i predicati di filtraggio (clausole **WHERE**). Una promettente linea di sviluppo consiste nell'ingegnerizzazione di un meccanismo di campionamento dinamico dei dati. All'atto della generazione del prompt, il sistema potrebbe interpellare direttamente il database target eseguendo query di tipo **SELECT DISTINCT** sulle colonne di tipo testuale (**VARCHAR** o **TEXT**) rilevanti. Valutando preventivamente la cardinalità e i valori effettivi di tali attributi, l'LLM verrebbe arricchito di una consapevolezza fattuale sul contenuto informativo del database. Questo approccio eliminerebbe la necessità di effettuare tentativi iterativi basati su valori errati o inesistenti, incrementando in modo significativo il tasso di successo esecutivo della query finale.

Bibliografia

- [1] OpenAI. Chatgpt. <https://chat.openai.com/>. Ultimo accesso: 26/08/2026.
- [2] OpenAI. Codex in chatgpt. <https://openai.com/codex/>. Ultimo accesso: 26/08/2026.
- [3] Google. Notebooklm / gemini notebook. <https://notebooklm.google/>. Ultimo accesso: 26/08/2026.
- [4] Google. Google gemini. <https://gemini.google.com/>. Ultimo accesso: 26/08/2026.
- [5] GeeksforGeeks. What are embeddings in machine learning? GeeksforGeeks, June 2024. Accessed: 2026-06-23.
- [6] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Improving text embeddings with large language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11897–11916, Bangkok, Thailand, August 2024. Association for Computational Linguistics.
- [7] Thinkia. Database vettoriali per rag: alimentare la ricerca semantica per llm. Thinkia Blog, September 2025. Accessed: 2026-06-23.
- [8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021.
- [9] Shailja Gupta, Rajesh Ranjan, and Surya Narayan Singh. A comprehensive survey of retrieval-augmented generation (rag): Evolution, current landscape and future directions, 2024.
- [10] Wikipedia contributors. Retrieval-augmented generation — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Retrieval-augmented_generation&oldid=1360040196, 2026. [Online; accessed 22-June-2026].
- [11] Michael Glass, Mustafa Eyceoz, Dharmashankar Subramanian, Gaetano Rossiello, Long Vu, and Alfio Gliozzo. Extractive schema linking for text-to-sql, 2025.
- [12] Maria Khalusova. Understanding embedding models: make an informed choice for your rag. Unstructured Blog, August 2024. Accessed: 2026-06-23.
- [13] Zijin Hong, Zheng Yuan, Qinggang Zhang, Hao Chen, Junnan Dong, Feiran Huang, and Xiao Huang. Next-generation database interfaces: A survey of llm-based text-to-sql, 2025.
- [14] A. Kanburoğlu and F. B. Tek. Text-to-SQL: A methodical review of challenges and models. *Turkish Journal of Electrical Engineering and Computer Sciences*, 32(3):403–419, 2024. DOI: <https://doi.org/10.55730/1300-0632.4077>.
- [15] Wenqiang Lei, Weixin Wang, Zhixin Ma, Tian Gan, Wei Lu, Min-Yen Kan, and Tat-Seng Chua. Re-examining the role of schema linking in text-to-SQL. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu, editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6943–6954, Online, November 2020. Association for Computational Linguistics.
- [16] Hanchen Xia, Feng Jiang, Naihao Deng, Cunxiang Wang, Guojiang Zhao, Rada Mihalcea, and Yue Zhang. r^3 : "this is my sql, are you with me?" a consensus-based multi-agent system for text-to-sql tasks, 2024.
- [17] Md Mahadi Hasan Nahid, Davood Rafiei, Weiwei Zhang, and Yong Zhang. Rethinking schema linking: A context-aware bidirectional retrieval approach for text-to-sql, 2026.

- [18] Dazhi Peng. X-sql: Expert schema linking and understanding of text-to-sql with multi-llms, 2025.
- [19] Yu Xia, Rui Wang, Xu Liu, Mingyan Li, Tong Yu, Xiang Chen, Julian McAuley, and Shuai Li. Beyond chain-of-thought: A survey of chain-of-x paradigms for llms, 2025.
- [20] Hossein A. Rahmani, Satyapriya Krishna, Xi Wang, Mohammadmehdi Naghiaei, and Emine Yilmaz. Self-correcting large language models: Generation vs. multiple choice, 2025.
- [21] SQLAI.ai. Sqlai.ai – generate sql queries in seconds for free. <https://www.sqlai.ai/>, Ultimo accesso: 26/06/2026.
- [22] Chat2DB. Chat2db – how to convert natural language to sql: A comprehensive step-by-step guide. <https://chat2db.ai/>, Ultimo accesso: 26/06/2026.
- [23] Vanna AI, Inc. Vanna.ai – open-source ai sql agent. <https://vanna.ai/>. Ultimo accesso: 26/06/2026.
- [24] AskYourDatabase. Askyourdatabase – ai-powered business intelligence chatbot. <https://www.askyourdatabase.com/>. Ultimo accesso: 26/06/2026.
- [25] Text2SQL. Text2sql – generate sql queries with ai. <https://text2sql.ai/>, Ultimo accesso: 26/06/2026.
- [26] AI2sql. Ai2sql – query builder. <https://www.ai2sql.io/>, Ultimo accesso: 26/06/2026.
- [27] BlazeSQL. Blazesql – turn questions into data insights, with sql ai. <https://www.blazesql.com/>, Ultimo accesso: 26/06/2026.
- [28] Formulabot. Formulabot. <https://www.formulabot.com/>, Ultimo accesso: 26/06/2026.
- [29] Sequel. Sequel – ai-powered sql generation. <https://www.sequel.sh/>, Ultimo accesso: 26/06/2026.
- [30] SQLChat. Sqlchat. <https://www.sqlchat.ai/>, Ultimo accesso: 26/06/2026.
- [31] Pietro Gasparini. Sorgente del codice del progetto di tesi. <https://github.com/Txt2SQL>, 2026.
- [32] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task, 2019.
- [33] Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C.C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23*, Red Hook, NY, USA, 2023. Curran Associates Inc.

Allegato A Prompt Ingegnerizzati

In questo allegato vengono riportati i testi integrali dei prompt di sistema implementati all'interno della classe `PromptBuilder` nel backend dell'applicazione. Questi costrutti rappresentano il nucleo logico della componente di *Prompt Engineering* della tesi. Ciascun prompt viene compilato dinamicamente a runtime iniettando le variabili contestuali estratte dagli orchestratori e dai *vector store*. I sorgenti aggiornati e i relativi metodi di assemblaggio sono consultabili nella repository ufficiale del progetto [31] all'interno del file `prompt_builder.py`.

Al fine di preservare l'esatta semantica interpretata dai modelli linguistici ed evitare alterazioni interpretative delle richieste di, tutti i prompt somministrati agli LLM agenti e valutatori sono stati ingegnerizzati in lingua inglese.

A.1 Prompt per la Generazione dello Schema

Il seguente prompt impiegato per istruire il modello linguistico a convertire le istruzioni DDL (Data Definition Language) fornite dall'utente in uno schema relazionale normalizzato in formato JSON.

Listing A.1: Struttura del prompt `schema_generation_prompt`

```
You are an expert database schema analyzer. Your task is to convert SQL DDL
statements into a structured JSON schema.

IMPORTANT:
- You MUST return ONLY valid JSON.
- The JSON must be syntactically correct (no missing commas, braces, or quotes).
- Every object and array must be properly closed.
- Do NOT include comments, code blocks, or explanations.

Required JSON format:
{
  "tables": [
    {
      "name": "table_name",
      "columns": [
        {"name": "column_name", "type": "SQL_TYPE", "constraints": ["PRIMARY KEY", "
          NOT NULL", ...]}
      ]
    }
  ]
}

Rules:
- Extract table names from CREATE TABLE statements
- Extract column names, types, and constraints
- Map SQL types directly (VARCHAR2 -> VARCHAR2, NUMBER -> NUMBER, etc.)
- Include constraints like PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT, REFERENCES
- For foreign keys, use "REFERENCES" constraint

SQL DDL to process:
""{raw_schema_text}""

Return ONLY the JSON object:
```

A.2 Prompt per l'Aggiornamento dello Schema

Qualora l'utente richieda una modifica o un'integrazione strutturale allo schema esistente, il metodo `schema_update_prompt` esegue un'operazione di fusione logica senza compromettere la conoscenza canonica precedentemente acquisita.

Listing A.2: Struttura del prompt `schema_update_prompt`

```
You are an expert database schema analyst.
```

```

You have been provided with:
1. The CURRENT canonical schema (JSON format)
2. NEW text describing additional tables or modifications to existing tables

Your task:
- Analyze the new text to identify any NEW tables or MODIFIED columns in existing tables
- Preserve all existing tables and columns from the current schema
- Add only the NEW tables or merge modifications into existing tables
- Return a SINGLE, complete JSON schema that includes both the current schema and the updates

IMPORTANT RULES:
- Do NOT remove any existing tables or columns
- If a table already exists, ADD new columns or UPDATE existing ones (don't duplicate )
- Maintain the same JSON structure: {"database": "<database_name>", "tables": [...], "semantic_notes": [...]}
- Return ONLY the updated JSON schema, no other text or explanations
- Each table must have: "name", "columns" (array)
- Each column must have: "name", "type", "constraints" (array)

=== CURRENT SCHEMA ===
{current_schema}

=== NEW TEXT ===
{raw_schema_text}

Return the UPDATED schema JSON:

```

A.3 Prompt per la Traduzione Text-to-SQL

Il prompt d'area `query_generation_prompt` costituisce il componente core del sistema di traduzione. Esso aggrega il contesto relazionale parziale, le richieste utente, i suggerimenti sulle chiavi esterne (*join hints*) e lo storico dei fallimenti passati, imponendo catene di vincoli stringenti per mitigare i fenomeni di allucinazione.

Listing A.3: Struttura del prompt `query_generation_prompt`

```

You are an expert SQL database assistant.
You will be provided with:
1. The partial description of the database schema (only the relevant tables)
2. The user's request in natural language.
3. Examples of previous SQL failures to correct

Your task is to return a single SQL query that satisfies the request,
using the provided tables and columns.

=== REQUEST ===
{user_request}

=== SCHEMA ===
{schema_context}

[Se presenti, il PromptBuilder inietta qui i JOIN PATH HINTS]

IMPORTANT CONSTRAINTS BASED ON PAST FAILURES:
- Do NOT use columns outside the schema
- Do NOT invent field or table names that don't exist.
- Always qualify columns when joining
- Do NOT use SELECT *
- Do NOT add WHERE clauses or conditions unless explicitly requested.
- Do NOT join tables unless necessary for the request.
- If using aggregates, include GROUP BY

```

```

- The sql query must be executable on a {engine.upper()} database.

[Se presenti, il PromptBuilder inietta qui la sezione PREVIOUS QUERY ERROR TO FIX]

Before writing the SQL query, internally determine:
- Which tables are required
- How they are joined
- Whether aggregation or grouping is required
- Which columns are selected

Do NOT output this reasoning.
Only output the final SQL query.

SQL QUERY (DO NOT ADD COMMENTS OR EXPLANATION TEXT BEFORE AND AFTER THE QUERY):

```

A.4 Prompt per il Feedback dei Circuiti di Valutazione

In questa sezione vengono riportati i prompt associati al modulo LLM Evaluator, impiegati per la diagnostica d'errore a runtime (A.4) e per la validazione semantica a ciclo chiuso dei record estratti (A.5).

Listing A.4: Struttura del prompt `explanation_prompt`.

```

You are an expert SQL debugger.
Explain why the following SQL query produced this runtime error.
Be concise and do NOT rewrite the query.

--- CONTEXT ---
{context}

--- SQL QUERY ---
{sql}

--- RUNTIME ERROR ---
{execution_output}

Provide a clear explanation of the cause.

```

Listing A.5: Struttura del prompt `evaluation_prompt`.

```

You are an expert SQL reviewer.

Your task is to evaluate whether the SQL query correctly answers
the user's request, based ONLY on the query results shown.

--- CONTEXT ---
{context}

--- USER REQUEST ---
{request}

--- SQL QUERY ---
{sql}

--- QUERY RESULT (first 20 rows) ---
{rows_text}

--- INSTRUCTIONS ---
Respond in EXACTLY one of the following formats:

1) If the query is correct:
CORRECT_QUERY

2) If the query is incorrect:
INCORRECT_QUERY: <clear explanation of what is wrong and how to fix it>

```

Rules:

- Do NOT rewrite the full SQL query.
- Be concise and precise.
- Judge correctness, not syntax or performance.

A.5 Prompt Ausiliari e di Benchmarking

Infine, si riportano i prompt secondari utilizzati rispettivamente per la classificazione euristica dell'input utente in fase di update (A.6) e per la validazione automatizzata dei risultati sperimentali a fronte dei dataset di benchmark (A.7).

Listing A.6: Struttura del prompt `update_classification_prompt`.

```
System: You are an assistant that classifies schema updates.
User: Text provided by the user:
{text}

Question: is this text
(A) a structural modification (addition or change of tables/columns/types)?
(B) a description or semantic note?
Answer only with "A" or "B".
```

Listing A.7: Struttura del prompt `llm_judge_prompt`.

```
You are judging whether a predicted SQL query should be considered correct for a
Spider-style text-to-SQL example.

Database id: {database_name}
Question: {question}

Gold query:
{gold_report.sql_code}

Gold query result:
{gold_report._format_query_session_result()}

Predicted query:
{pred_report.sql_code}

Predicted query result:
{pred_report._format_query_session_result()}

Decide whether the predicted query is semantically correct relative to the gold query
and the observed execution results.

Return JSON only in this format:
{"verdict":"correct"|"incorrect","reason":"short explanation"}
```

Allegato B Video Interfaccia

<https://github.com/Txt2SQL/Back-End/blob/main/Preview.mp4>

Allegato C Tabelle Statistiche

In questo allegato vengono riportate le tabelle statistiche relative ai modelli linguistici (*Large Language Models*, LLM) sottoposti a scrutinio durante la fase di *testing* e *benchmarking*. I dati raccolti abilitano un'analisi comparativa approfondita e quantitativa delle prestazioni di ciascun modello a fronte di diversi contesti operativi. Per ogni LLM esaminato sono strutturate tre tipologie di tabelle:

1. **Tabella di Status:** Offre un quadro olistico sugli esiti generativi, isolando le metriche di successo, i tempi di calcolo medi (*execution latency*), il numero di tentativi richiesti e la tassonomia degli errori riscontrati (sintattici o di runtime).
2. **Tabella delle Correlazioni sui Tentativi (Attempts Correlations):** Analizza la dipendenza statistica tra i tentativi andati a buon fine e la distribuzione del numero di iterazioni effettuate nel ciclo chiuso.
3. **Tabella delle Correlazioni sulla Complessità (Complexity Correlations):** Evidenzia la reattività del modello al crescere della complessità intrinseca della query SQL target.

Tutte le metriche di correlazione sono calcolate mediante i coefficienti di Pearson e Spearman, corredati dal relativo *p-value* per attestarne la significatività statistica. I dataset originari in formato grezzo sono archiviati come file `.csv` nella repository ufficiale del progetto [31].

C.1 Tabelle di GPT-4o

C.1.1 Tabella Status

https://github.com/Txt2SQL/Back-End/blob/main/tests/output/stats/gpt-4o_report/status.csv

C.1.2 Tabella Attempts Correlations

https://github.com/Txt2SQL/Back-End/blob/main/tests/output/stats/gpt-4o_report/attempts_correlations.csv

C.1.3 Tabella Complexity Correlations

https://github.com/Txt2SQL/Back-End/blob/main/tests/output/stats/gpt-4o_report/complexity_correlations.csv

C.2 Tabelle di GPT-5-mini

C.2.1 Tabella Status

https://github.com/Txt2SQL/Back-End/blob/main/tests/output/stats/gpt-5-mini_report/complexity_correlations.csv

C.2.2 Tabella Attempts Correlations

https://github.com/Txt2SQL/Back-End/blob/main/tests/output/stats/gpt-5-mini_report/attempts_correlations.csv

C.2.3 Tabella Complexity Correlations

https://github.com/Txt2SQL/Back-End/blob/main/tests/output/stats/gpt-5-mini_report/complexity_correlations.csv

C.3 Tabelle di Codellama

C.3.1 Tabella Status

https://github.com/Txt2SQL/Back-End/blob/main/tests/output/stats/codellama_34b_report/status.csv

C.3.2 Tabella Attempts Correlations

https://github.com/Txt2SQL/Back-End/blob/main/tests/output/stats/codellama_34b_report/attempts_correlations.csv

C.3.3 Tabella Complexity Correlations

https://github.com/Txt2SQL/Back-End/blob/main/tests/output/stats/codellama_34b_report/complexity_correlations.csv

C.4 Tabelle di Codestral

C.4.1 Tabella Status

https://github.com/Txt2SQL/Back-End/blob/main/tests/output/stats/codestral_22b_report/status.csv

C.4.2 Tabella Attempts Correlations

https://github.com/Txt2SQL/Back-End/blob/main/tests/output/stats/codestral_22b_report/attempts_correlations.csv

C.4.3 Tabella Complexity Correlations

https://github.com/Txt2SQL/Back-End/blob/main/tests/output/stats/codestral_22b_report/complexity_correlations.csv

C.5 Tabelle di SQLCoder

C.5.1 Tabella Status

https://github.com/Txt2SQL/Back-End/blob/main/tests/output/stats/sqlcoder_34b_report/status.csv

C.5.2 Tabella Attempts Correlations

https://github.com/Txt2SQL/Back-End/blob/main/tests/output/stats/sqlcoder_34b_report/attempts_correlations.csv

C.5.3 Tabella Complexity Correlations

https://github.com/Txt2SQL/Back-End/blob/main/tests/output/stats/sqlcoder_34b_report/complexity_correlations.csv

C.6 Tabelle di Qwen3-Coder-Next

C.6.1 Tabella Status

https://github.com/Txt2SQL/Back-End/blob/main/tests/output/stats/Qwen3-coder-next_report/status.csv

C.6.2 Tabella Attempts Correlations

https://github.com/Txt2SQL/Back-End/blob/main/tests/output/stats/Qwen3-coder-next_report/attempts_correlations.csv

C.6.3 Tabella Complexity Correlations

https://github.com/Txt2SQL/Back-End/blob/main/tests/output/stats/Qwen3-coder-next_report/complexity_correlations.csv

C.7 Tabella Database

Questa tabella contiene in maniera riassuntiva i risultati ottenuti a livello di database. L'obiettivo di questa tabella è quello di aggregare i punteggi sulla complessità dei singoli database, sul piano della query e sul piano dello schema, mostrando il livello di difficoltà complessivo di ciascun database e la sua influenza sulle prestazioni dei modelli linguistici.

https://github.com/Txt2SQL/Back-End/blob/main/tests/output/stats/databases_report/summary.csv